UNIVERSIDAD EL BOSQUE

AVANCE 1 — NÚCLEO TRANSACCIONAL EN POSTGRESQL

CINE AURORA



TAMAÑO DE GRUPO:

2 INTEGRANTES

INTEGRANTES:

YEISON ALEJANDRO SILVA RODRIGUEZ
LEIDY VANESSA GIL DAZA

BASES DE DATOS 2

DOCENTE:
FRANCISCO BARGUIL

15 DE OCTUBRE 2026
BOGOTA D.C

1. Identidad de la empresa de cine

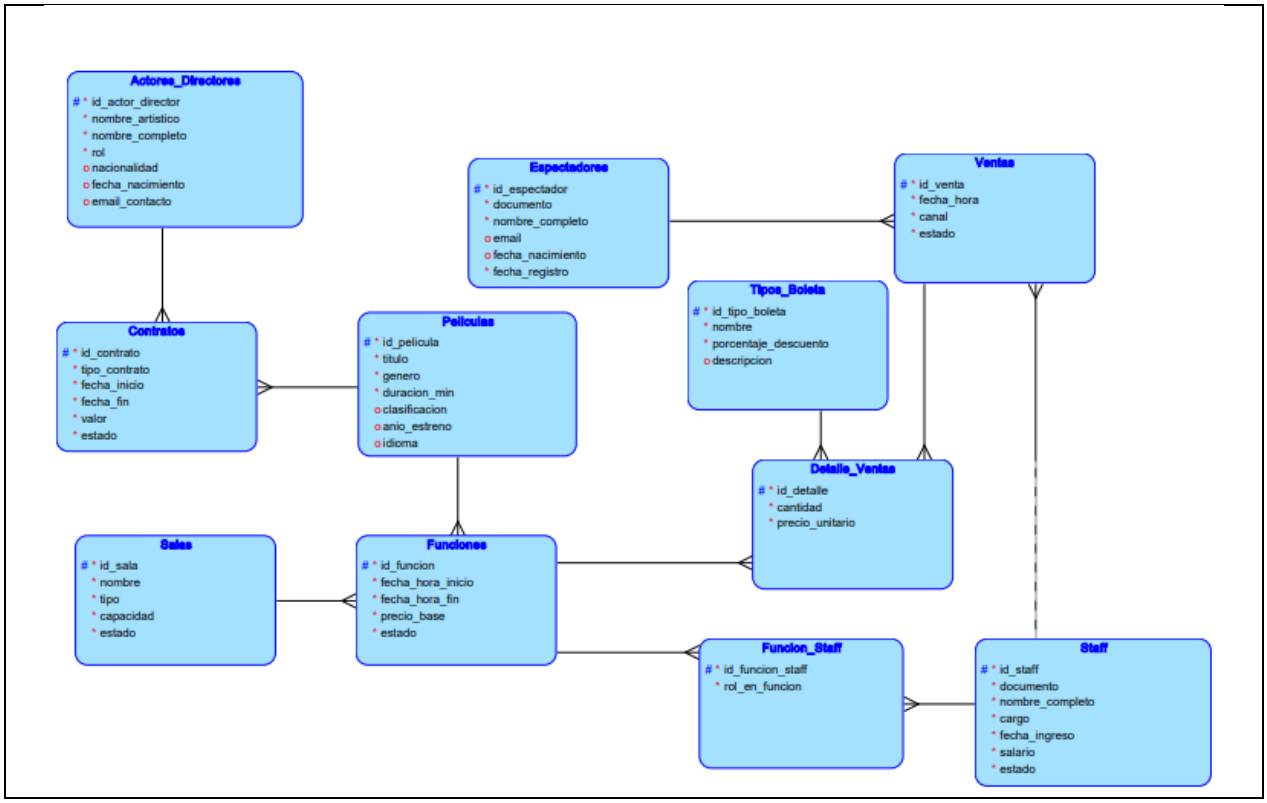
Cine Aurora (grupo Cinemas 8) es una empresa de cine de Bogotá. Pasamos películas de cine de autor, clásico e independiente y también algunos estrenos de aventura y ciencia ficción. Además, hacemos presentaciones, charlas y ciclos con actores y directores, por eso la base de datos maneja también los contratos con ellos y no solo la venta de boletas.

Nombre	**Cine Aurora**
Enfoque de programación	Cine de autor, clásico e independiente (drama, documental, animación, romance y musical), con algunos estrenos de aventura, terror y ciencia ficción. También hay presentaciones, charlas y ciclos con actores y directores.
Ciudad	Bogotá D.C., Colombia
Modalidad de operación	Mixta es decir que se vende en la taquilla y también por internet.
Capacidad estimada	1.200 espectadores, que es la suma de la capacidad de las 8 salas
Número de salas	8 salas: 2 VIP (Aurora de 80 y Boreal de 100), 2 en 3D (Eclipse de 150 y Fénix de 160) y 4 en 2D (Cenit y Delta de 120, Galaxia de 220 y Horizonte de 250)
Actores/directores registrados	40 artistas en la base de datos.
Particularidades del negocio	-Tenemos tres tipos de salas (2D, 3D y VIP), y cada una maneja un precio base para sus funciones. -Las boletas se dividen en 4 categorías: General (sin descuento), Estudiante (20% de descuento), Tercera edad (30% de descuento) y Matiné (25% de descuento). -Los contratos con los actores y directores pueden ser de tres tipos: Presentación, Charla o Ciclo. -Las ventas presenciales en la taquilla siempre van asociadas a un taquillero, mientras que las compras online se hacen de forma directa sin requerir personal de taquilla. -A cada función se le asigna personal de staff cumpliendo roles específicos: responsable, proyección o acomodación. -El sistema cuenta con validaciones estrictas para evitar sobreventas (no se pueden vender más boletas de la capacidad de la sala) y para impedir que se crucen dos funciones al mismo tiempo en una misma sala.
Logo	

2. Modelo entidad-relación

Tenemos las 8 entidades que pide el avance (Actores_Directores, Contratos, Salas, Funciones, Tipos_Boleta, Ventas, Espectadores y Staff). Para poder llegar a la tercera forma normal tuvimos que agregar la tabla películas, y para las relaciones de muchos a muchos creamos dos tablas intermedias: detalle_ventas y funcion_staff. La tabla contratos también cumple ese papel, porque une a los actores y directores con las películas.

Figura 1. Diagrama entidad-relación



2.1 Relaciones y cardinalidades

Relación	Cardinalidad	Descripción
actores_directores - contratos	1: N	Un actor o director puede tener varios contratos, pero cada contrato es de un solo artista.
películas - contratos	1: N	Una película puede tener varios contratos.
salas - funciones	1: N	Una sala tiene muchas funciones y cada función es en una sola sala.
películas - funciones	1: N	Una película se pasa en muchas funciones.
espectadores - ventas	1: N	Un espectador puede hacer muchas ventas.
staff - ventas	0..1: N	Un taquillero registra muchas ventas en la taquilla. Las ventas en línea no tienen taquillero.
ventas - detalle_ventas	1: N	Una venta tiene una o más líneas de detalle.

Relación	Cardinalidad	Descripción
funciones - detalle_ventas	1: N	Una función puede aparecer en muchas líneas de venta.
tipos_boleta - detalle_ventas	1: N	Un tipo de boleta se usa en muchas líneas de venta.
funciones - funcion_staff	1: N	Una función tiene varios empleados asignados.
staff - funcion_staff	1: N	Un empleado trabaja en muchas funciones.

2.2 Relaciones muchos a muchos resueltas

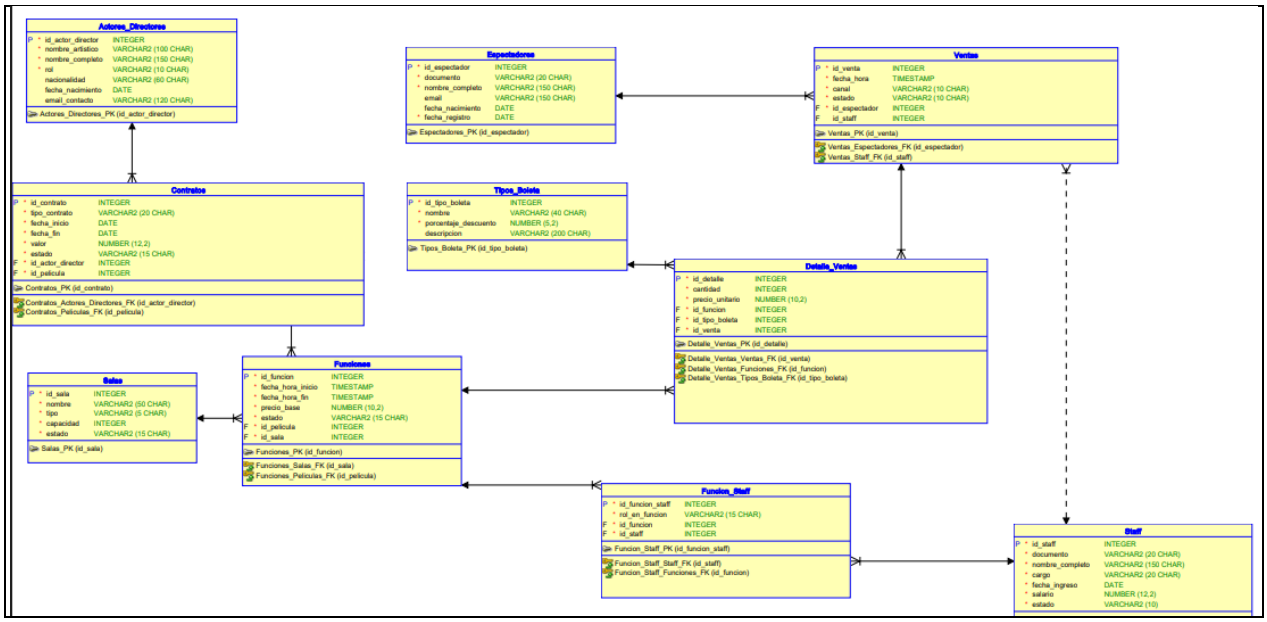
- actores_directores y películas, por medio de contratos con tipo, fechas, valor y estado.
- ventas y funciones, por medio de detalle_ventas con el tipo de boleta, la cantidad y el precio unitario
- funciones y staff, por medio de funcion_staff con el rol del empleado en la función.

3. Modelo relacional (3FN)

PK es llave primaria, FK es llave foránea y UQ es valor único.

Relación	Atributos	Llaves
actores_directores	id_actor_director, nombre_artistico, nombre_completo, rol, nacionalidad, fecha_nacimiento, email_contacto	PK: id_actor_director
peliculas	id_pelicula, titulo, genero, duracion_min, clasificacion, anio_estreno, idioma	PK: id_pelicula
salas	id_sala, nombre, tipo, capacidad, estado	PK: id_sala; UQ: nombre
staff	id_staff, documento, nombre_completo, cargo, fecha_ingreso, salario, estado	PK: id_staff; UQ: documento
tipos_boleta	id_tipo_boleta, nombre, porcentaje_descuento, descripcion	PK: id_tipo_boleta; UQ: nombre
espectadores	id_espectador, documento, nombre_completo, email, fecha_nacimiento, fecha_registro	PK: id_espectador; UQ: documento
contratos	id_contrato, id_actor_director, id_pelicula, tipo_contrato, fecha_inicio, fecha_fin, valor, estado	PK: id_contrato; FK: id_actor_director, id_pelicula
funciones	id_funcion, id_sala, id_pelicula, fecha_hora_inicio, fecha_hora_fin, precio_base, estado	PK: id_funcion; FK: id_sala, id_pelicula
ventas	id_venta, id_espectador, id_staff, fecha_hora, canal, estado	PK: id_venta; FK: id_espectador, id_staff (opcional)
detalle_ventas	id_detalle, id_venta, id_funcion, id_tipo_boleta, cantidad, precio_unitario	PK: id_detalle; UQ: (id_venta, id_funcion, id_tipo_boleta); FK: id_venta, id_funcion, id_tipo_boleta
funcion_staff	id_funcion_staff, id_funcion, id_staff, rol_en_funcion	PK: id_funcion_staff; UQ: (id_funcion, id_staff); FK: id_funcion, id_staff

Figura 2. Modelo relacional



3.1 Justificación de la normalización

Primera forma normal (1FN): todos los campos tienen un solo valor, no hay listas ni columnas repetidas. Por ejemplo, las líneas de una venta no están como columnas dentro de ventas, sino como filas en detalle_ventas.

Segunda forma normal (2FN): todas las tablas tienen una llave primaria de un solo campo (un id que se genera solo), así que no hay dependencias parciales. En las tablas intermedias además dejamos un valor único sobre la combinación (por ejemplo, id_venta, id_funcion e id_tipo_boleta) para que no se repitan líneas.

Tercera forma normal (3FN): ningún campo que no sea llave depende de otro campo que tampoco lo sea. Por ejemplo:

- Los datos de la película (título, género, duración, clasificación) dependen de la película y no de la función ni del contrato, por eso están en películas.
- El porcentaje de descuento depende del tipo de boleta, entonces está en tipos_boleta y no se repite en cada venta.
- La capacidad y el tipo de sala dependen de la sala y no de la función, por eso están en salas.
- Los datos del espectador y del empleado están en sus propias tablas y las ventas solo los llaman con una llave foránea.

3.2 Decisiones de diseño sobre datos derivados

- precio_unitario (en detalle_ventas) lo guardamos como dato histórico, porque es lo que realmente se cobró y no debería cambiar si después cambia el precio base o el descuento. Lo calcula el trigger 1. El total de una línea se saca con cantidad por precio_unitario y no se guarda.

- fecha_hora_fin (en funciones) se podría calcular con la hora de inicio más la duración de la película, pero la dejamos guardada para poder revisar los cruces de horario. La calcula el trigger 4, así que no queda con un valor incorrecto.

4. Script SQL de creación

El archivo 01_creacion_tablas.sql crea las 11 tablas con sus llaves primarias, llaves foráneas, restricciones (UNIQUE, NOT NULL, DEFAULT y CHECK) e índices básicos. Empieza borrando las tablas si ya existen (DROP TABLE IF EXISTS ... CASCADE), así que se puede volver a ejecutar desde cero.

4.1 Restricciones de integridad por tabla

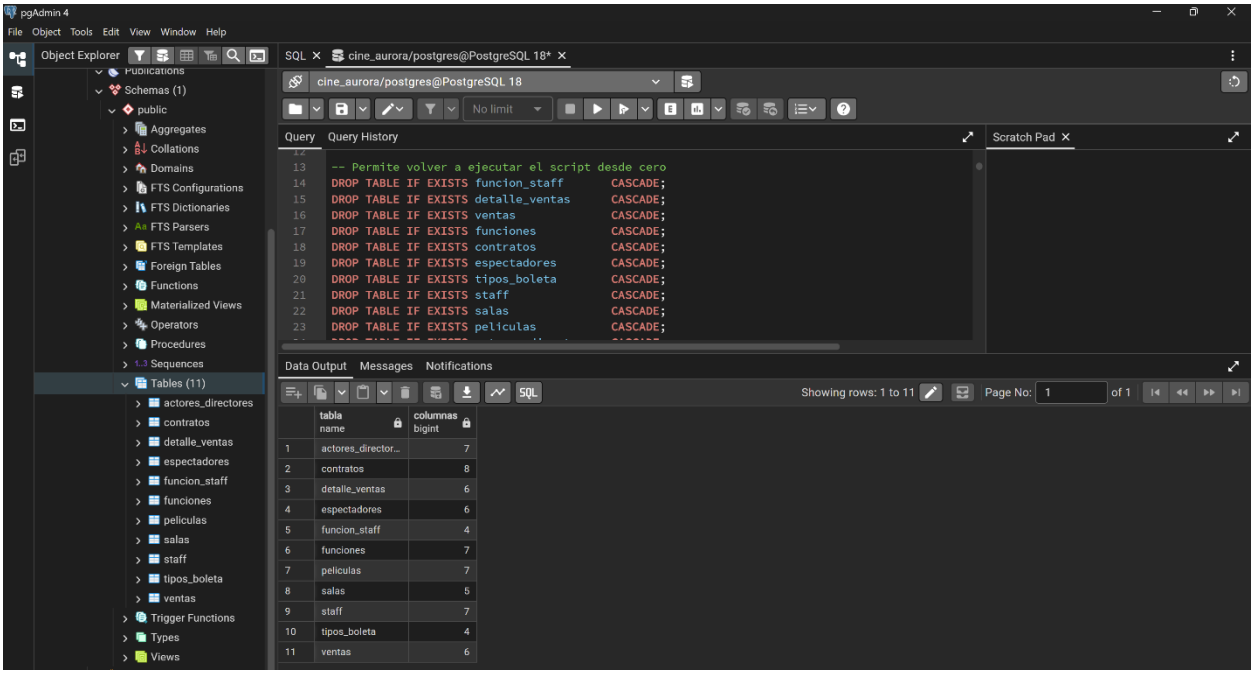
Tabla	Restricciones principales
actores_directores	El rol solo puede ser Actor, director o Ambos; los nombres no pueden estar vacíos; la fecha de nacimiento debe ser desde 1900; el correo debe tener formato válido.
peliculas	El título no puede estar vacío; la duración va entre 1 y 600 minutos; el año de estreno va entre 1888 y 2100.
salas	El nombre es único; el tipo solo puede ser 2D, 3D o VIP; la capacidad va entre 1 y 250; el estado es Activa o Mantenimiento.
staff	El documento es único; el cargo es Taquillero, Acomodador, Proyeccionista o Gerente; el salario debe ser mayor que 0; el estado es Activo o Inactivo; la fecha de ingreso debe ser desde 1990.
tipos_boleta	El nombre es único; el porcentaje de descuento va entre 0 y 100.
espectadores	El documento es único; el nombre no puede estar vacío; el correo debe ser válido; la fecha de nacimiento debe ser desde 1900.
contratos	El tipo es Presentación, Charla o Ciclo; la fecha de fin no puede ser antes de la de inicio; el valor no puede ser negativo; el estado es Vigente, Finalizado o Cancelado; llaves foráneas a actores_directores y películas.
funciones	La hora de fin debe ser mayor que la de inicio; el precio base no puede ser negativo; el estado es Programada, En curso, Finalizada o Cancelada; llaves foráneas a salas y películas.
ventas	El canal es Presencial u Online; el estado es Pagada o Anulada; una venta presencial siempre tiene empleado; llaves foráneas a espectadores y staff.
detalle_ventas	La cantidad debe ser mayor que 0; el precio unitario no puede ser negativo; no se repite la misma línea (venta, función y tipo de boleta); llaves foráneas a ventas, funciones y tipos_boleta.
funcion_staff	El rol es responsable, Proyección o Acomodación; un empleado no se repite en la misma función; llaves foráneas a funciones y staff.

4.2 Índices básicos

- Índices únicos sobre (email) en espectadores y actores_directores, para que no se repita un correo, aunque cambien las mayúsculas.

- idx_funciones_sala_inicio (id_sala, fecha_hora_inicio), que se usa para revisar los cruces de horario de una sala.
- Índices sobre las llaves foráneas de ventas, detalle_ventas, contratos, funciones y funcion_staff, para que los reportes y la validación de capacidad sean más rápidos.

Captura 1. Ejecución del script 01_creacion_tablas.sql

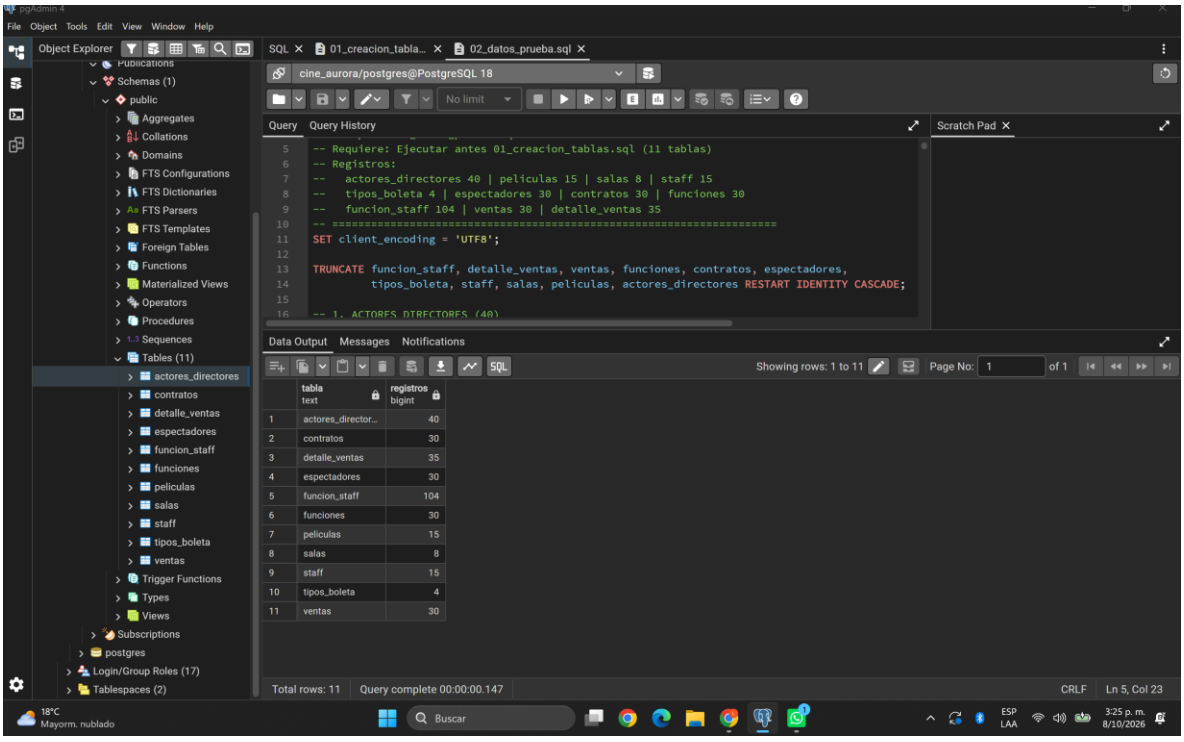


5. Datos de prueba

El archivo 02_datos_prueba.sql se ejecuta después del 01. Primero limpia las tablas y después inserta los datos, las ventas apuntan a espectadores que existen, las funciones tienen película y sala válidas, el detalle de ventas tiene el precio con el descuento del tipo de boleta y cada función tiene su personal asignado.

Tabla	Registros
actores_directores	40
espectadores	30
ventas	30
funciones	30 (20 finalizadas y 10 programadas)
películas	15
salas	8
staff	15
tipos_boleta	4
contratos	30
funcion_staff	104
detalle_ventas	35

Captura 2. Ejecución del script 02_datos_prueba.sql



6. Triggers

El archivo 03_triggers.sql tiene 8 triggers en 7 tablas, 6 de las 8 tablas pedidas (detalle_ventas, ventas, funciones, contratos, salas, staff y espectadores), así que la validación no queda toda en una sola tabla. Cada trigger tiene su función en PL/pgSQL y se crea con DROP TRIGGER IF EXISTS para poder ejecutar el script otra vez.

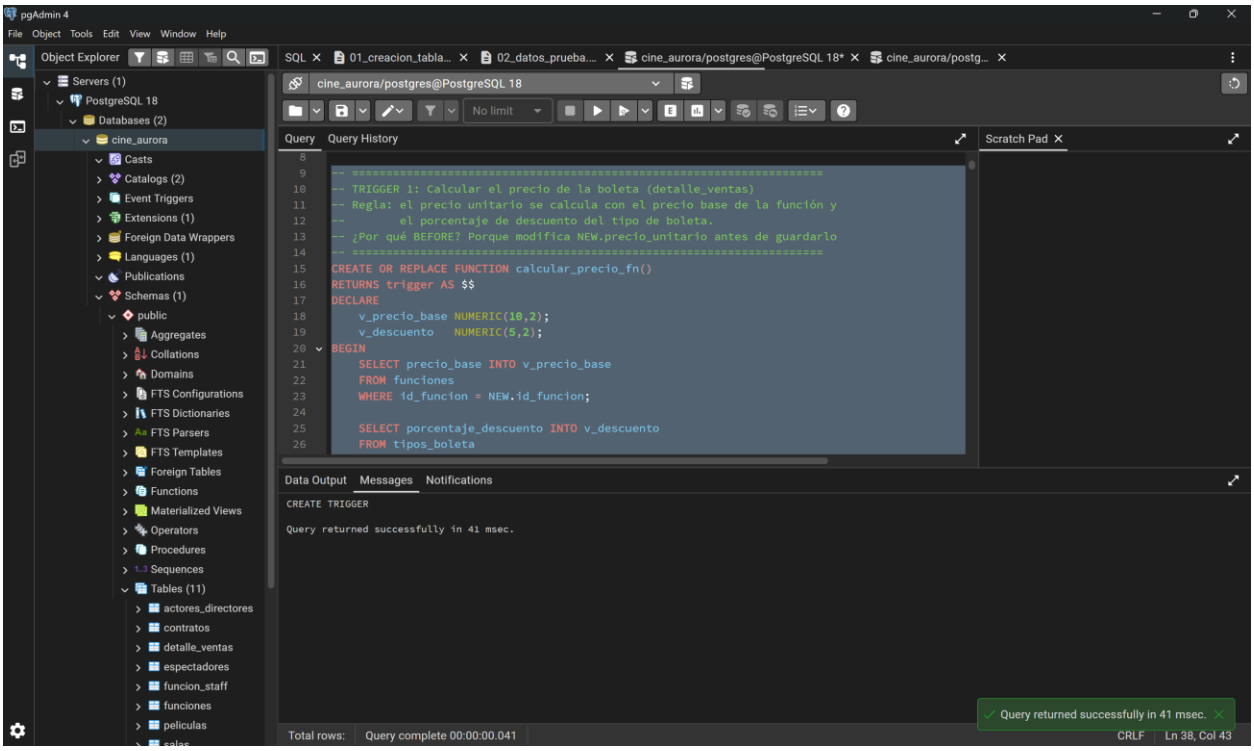
	Trigger	Tabla	Evento
1	trg_calcular_precio	detalle_ventas	BEFORE INSERT
2	trg_validar_capacidad	detalle_ventas	BEFORE INSERT OR UPDATE
3	trg_validar_taquillero	ventas	BEFORE INSERT
4	trg_validar_horario_funcion	funciones	BEFORE INSERT OR UPDATE
5	trg_validar_contrato	contratos	BEFORE INSERT OR UPDATE
6	trg_validar_mantenimiento_sala	salas	BEFORE UPDATE
7	trg_validar_inactivar_staff	staff	BEFORE UPDATE
8	trg_normalizar_espectador	espectadores	BEFORE INSERT OR UPDATE

Trigger 1: trg_calcular_precio

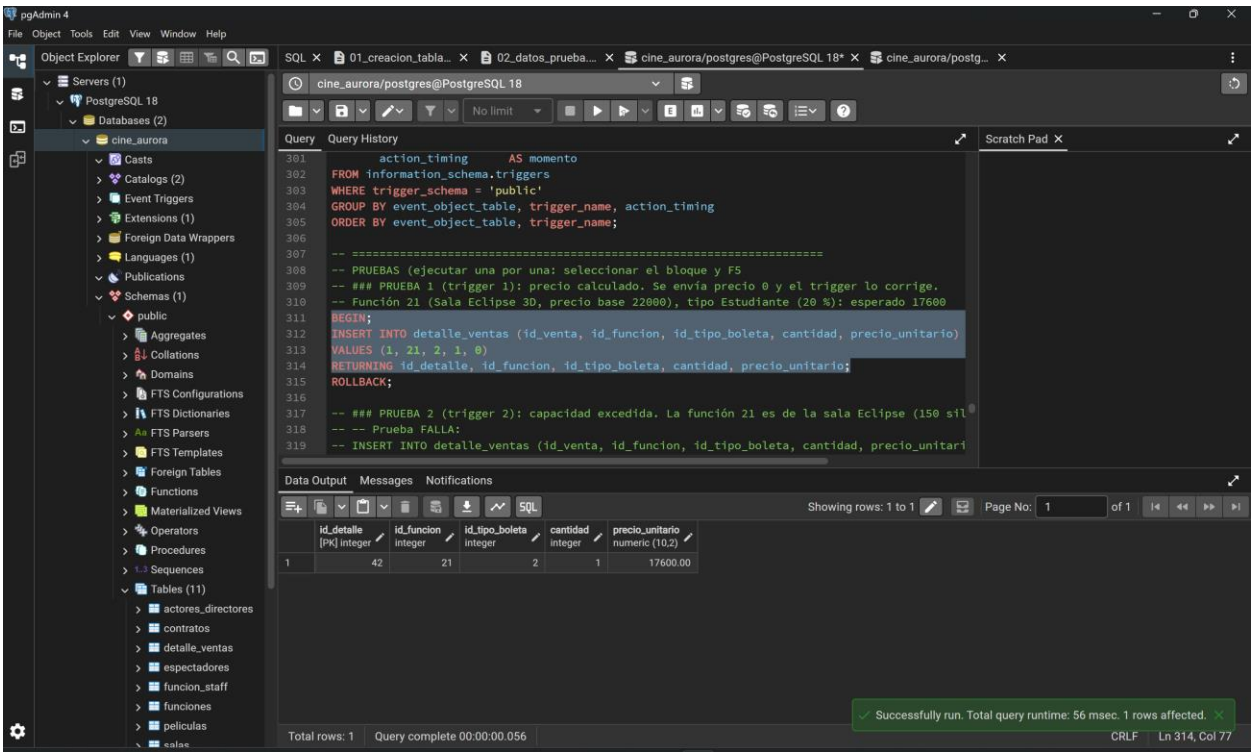
Tabla sobre la que actúa	detalle_ventas
Evento	BEFORE INSERT
Función asociada	calcular_precio_fn()
Regla de negocio que protege	El precio de la boleta siempre es el precio base de la función menos el descuento del tipo de boleta. Nadie puede escribir el precio a mano.

Busca el precio_base de la función y el porcentaje de descuento del tipo de boleta, y guarda en NEW.precio_unitario el precio base por (1 - descuento/100). Así no se pueden cobrar valores incorrectos.

Captura 3. Ejecución del trigger 1 (trg_calcular_precio)



Captura 4. Prueba del trigger 1 (trg_calcular_precio)



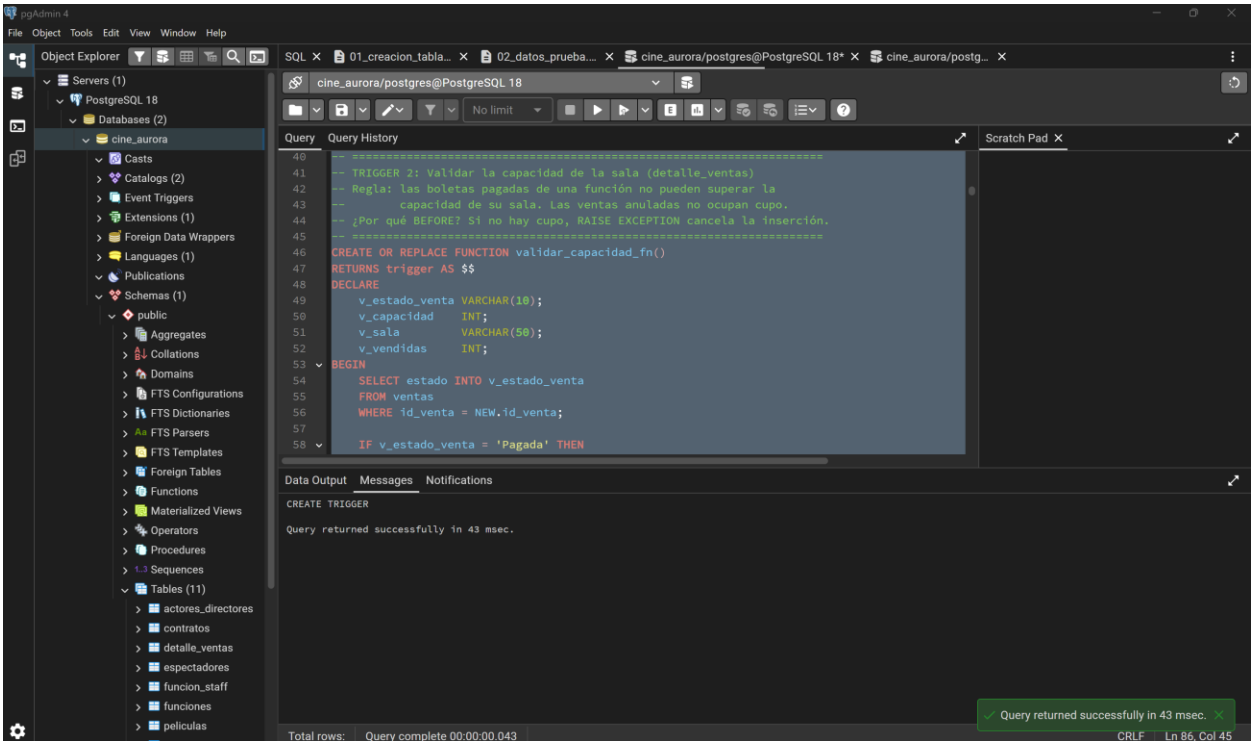
Trigger 2: trg_validar_capacidad

Tabla sobre la que actúa	detalle_ventas

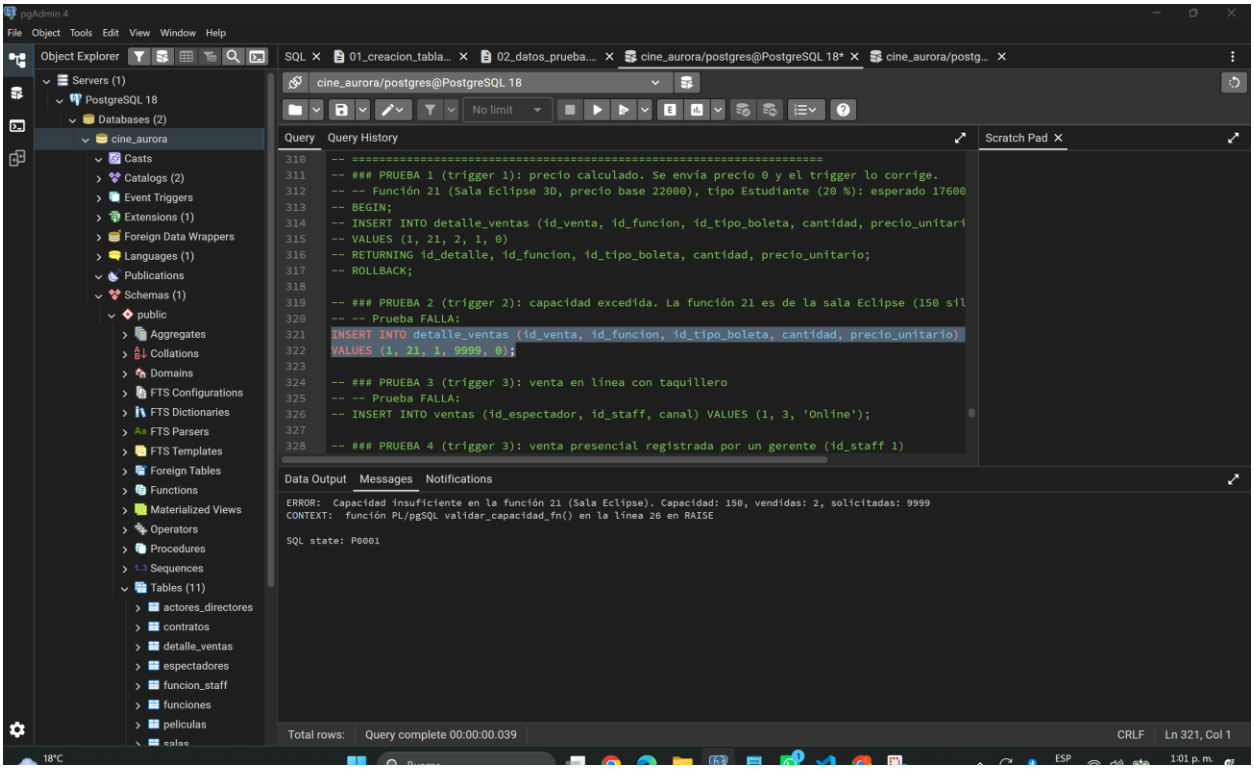
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_capacidad_fn()
Regla de negocio que protege	No se pueden vender más boletas que sillas tiene la sala.

Si la venta está Pagada, suma las boletas pagadas que ya tiene esa función, le suma las nuevas y las compara con la capacidad de la sala. Las ventas anuladas no cuentan, porque esas sillas quedan libres. Si se pasa de la capacidad, lanza un RAISE EXCEPTION con el cupo que queda.

Captura 6. Ejecución del trigger 2 (trg_validar_capacidad)



Captura 7. Prueba del trigger 2 (trg_validar_capacidad)

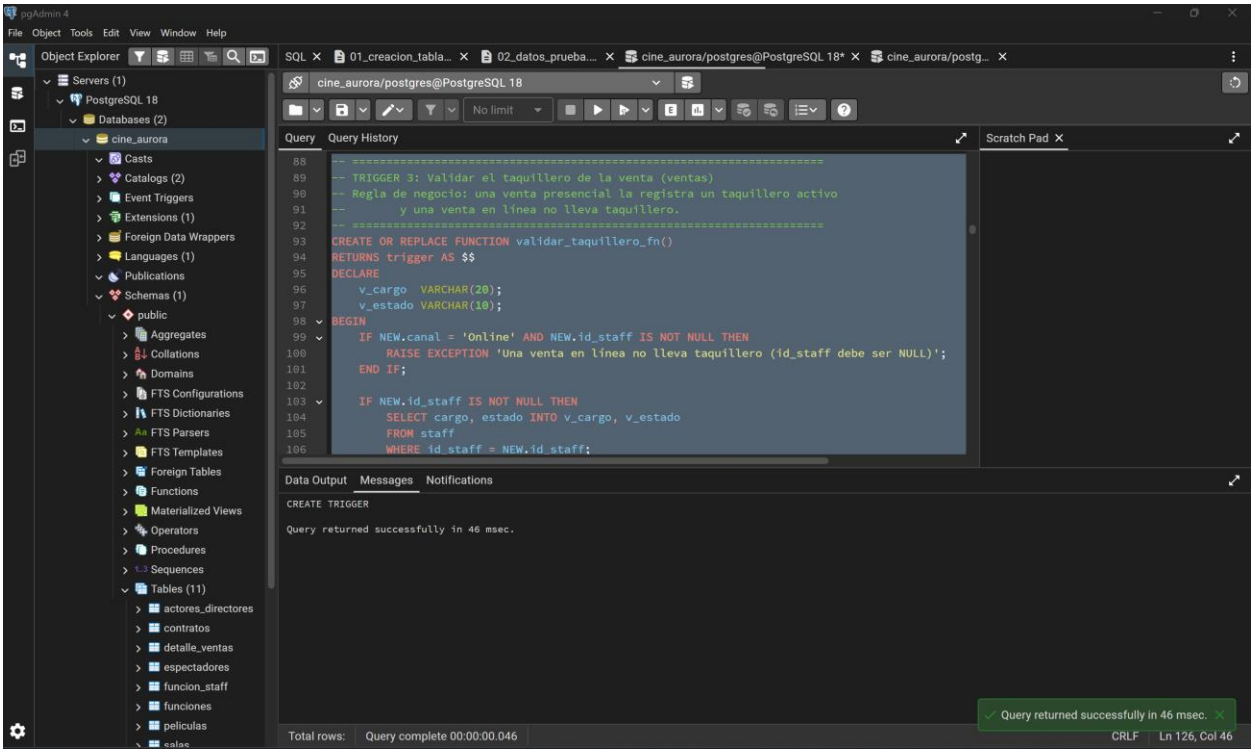


Trigger 3: trg_validar_taquillero

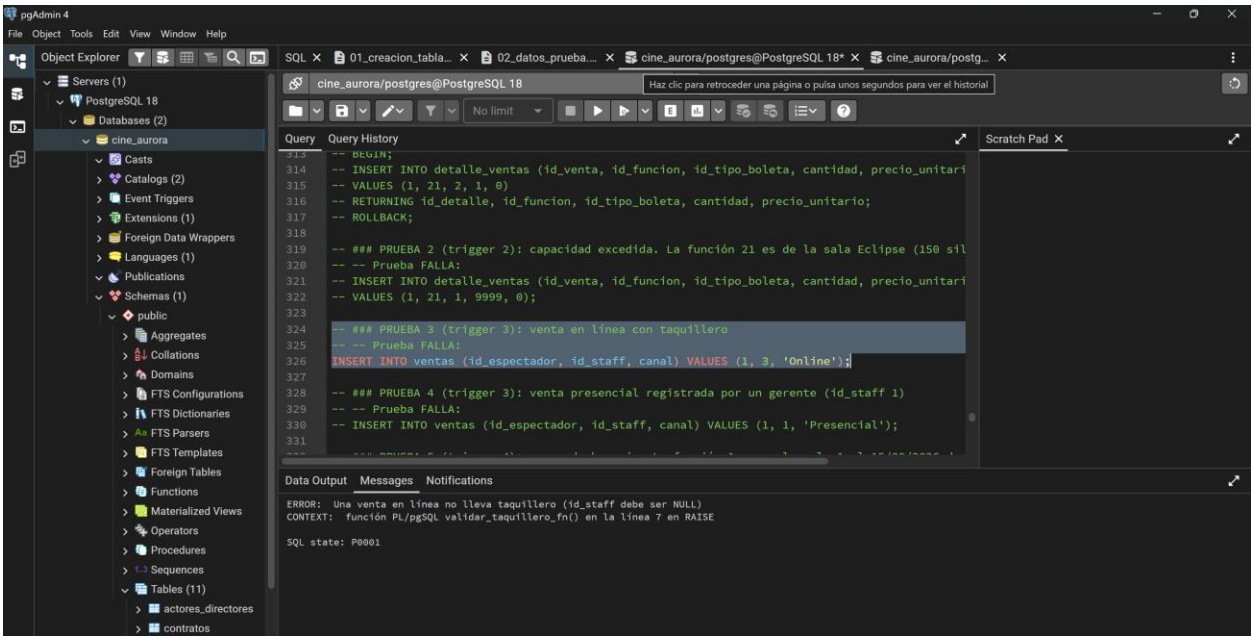
Tabla sobre la que actúa	ventas
Evento	BEFORE INSERT
Función asociada	validar_taquillero_fn()
Regla de negocio que protege	Una venta en línea no lleva empleado, y si lleva empleado tiene que ser un taquillero activo.

Rechaza las ventas Online que traen id_staff. Cuando la venta sí trae empleado, revisa que su cargo sea Taquillero y que esté Activo. Por ejemplo, un gerente o un empleado inactivo no puede vender en la taquilla.

Captura 8. Ejecución del trigger 3 (trg_validar_taquillero)



Captura 9. Prueba del trigger 3 (trg_validar_taquillero)

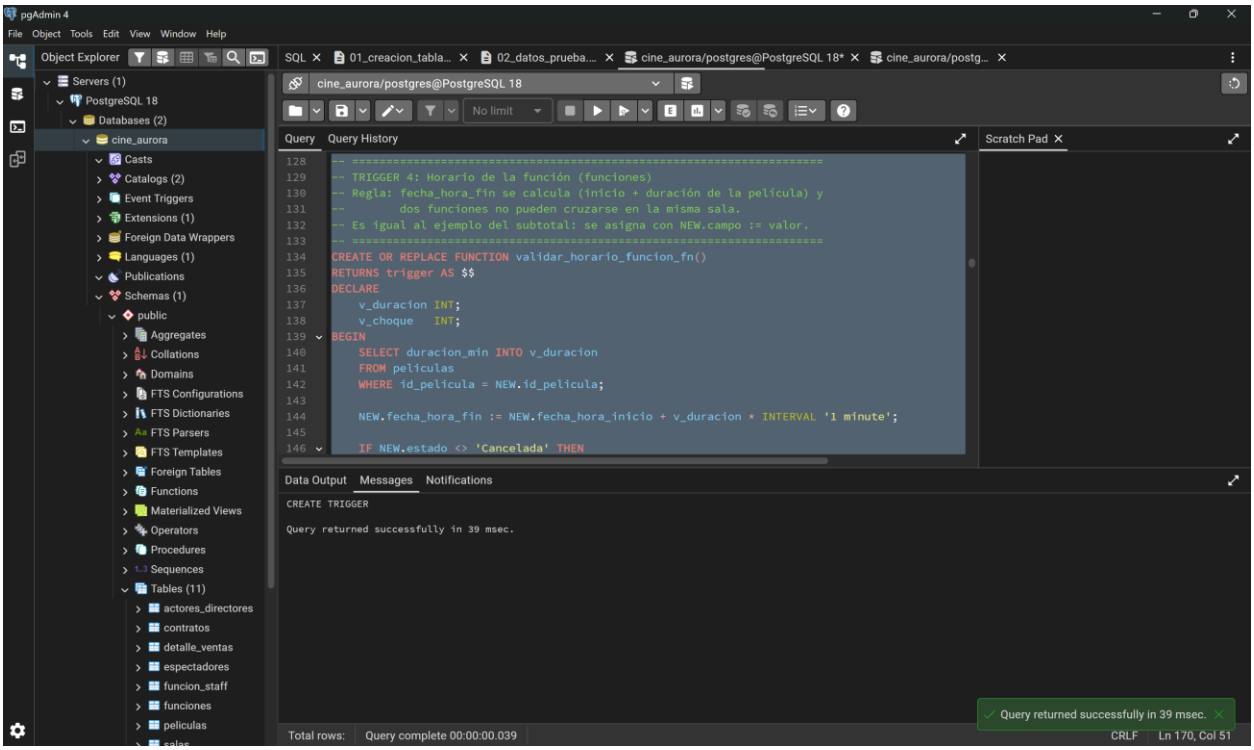


Trigger 4: trg_validar_horario_funcion

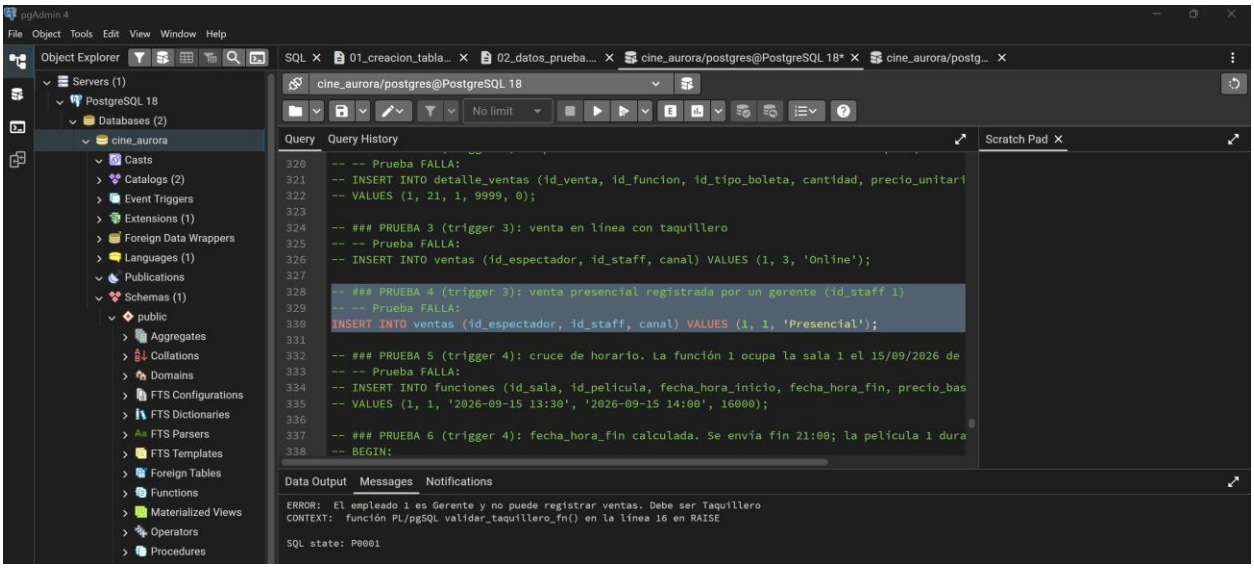
Tabla sobre la que actúa	funciones
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_horario_funcion_fn()
Regla de negocio que protege	La hora de fin es la hora de inicio más la duración de la película, y dos funciones no se pueden cruzar en la misma sala.

Calcula NEW.fecha_hora_fin con la duración de la película. Si la función no está cancelada, busca otra función no cancelada en la misma sala que se cruce en horario, y si la encuentra lanza un RAISE EXCEPTION diciendo con cuál choca.

Captura 10. Ejecución del trigger 4 (trg_validar_horario_funcion)



Captura 11. Prueba del trigger 4 (trg_validar_horario_funcion)



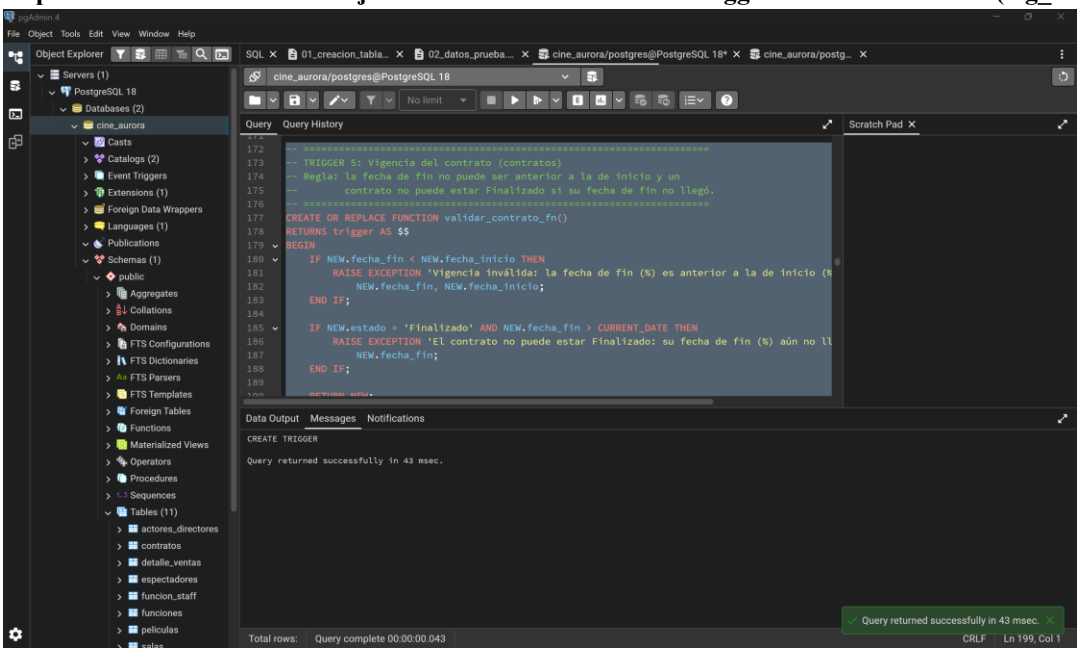
Trigger 5: trg_validar_contrato

Tabla sobre la que actúa	contratos

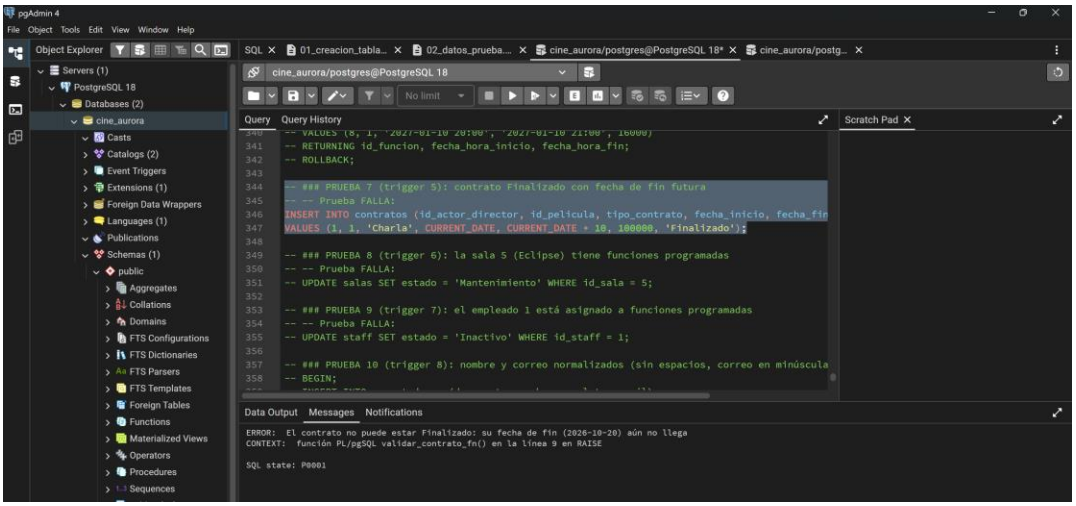
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_contrato_fn()
Regla de negocio que protege	Un contrato no puede terminar antes de empezar, ni estar Finalizado si su fecha de fin todavía no llega.

Revisa que las fechas del contrato tengan sentido y que el estado corresponda con la fecha de hoy, para que no queden contratos con un estado que no es real.

Captura 12. Ejecución del trigger 5 (trg_validar_contrato)



Captura 13. Prueba del trigger 5 (trg_validar_contrato)

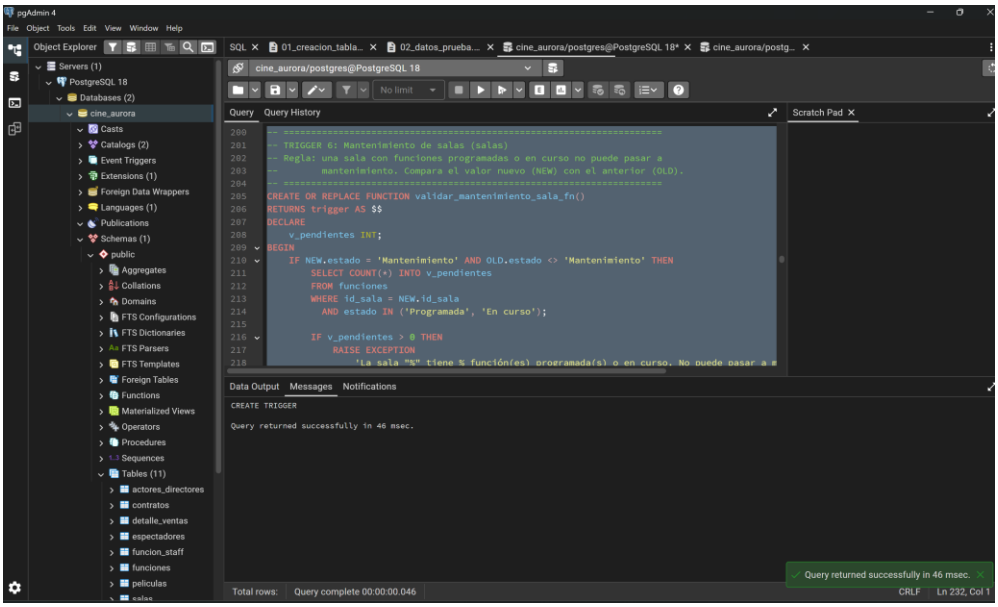


Trigger 6: trg_validar_mantenimiento_sala

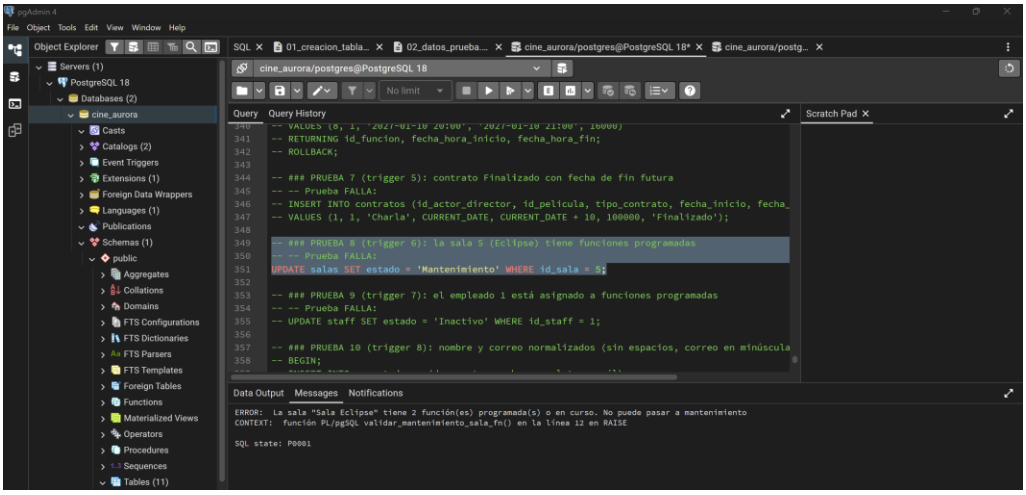
Tabla sobre la que actúa	salas
Evento	BEFORE UPDATE
Función asociada	validar_mantenimiento_sala_fn()
Regla de negocio que protege	Una sala no puede pasar a Mantenimiento si todavía tiene funciones programadas o en curso.

Usa OLD y NEW para revisar solo cuando el estado cambia a Mantenimiento. Cuenta las funciones Programadas o En curso de esa sala y, si hay alguna, lanza un RAISE EXCEPTION. Así ninguna función se queda sin sala.

Captura 14. Ejecución del trigger 6 (trg_validar_mantenimiento_sala)



Captura 15. Prueba del trigger 6 (trg_validar_mantenimiento_sala)

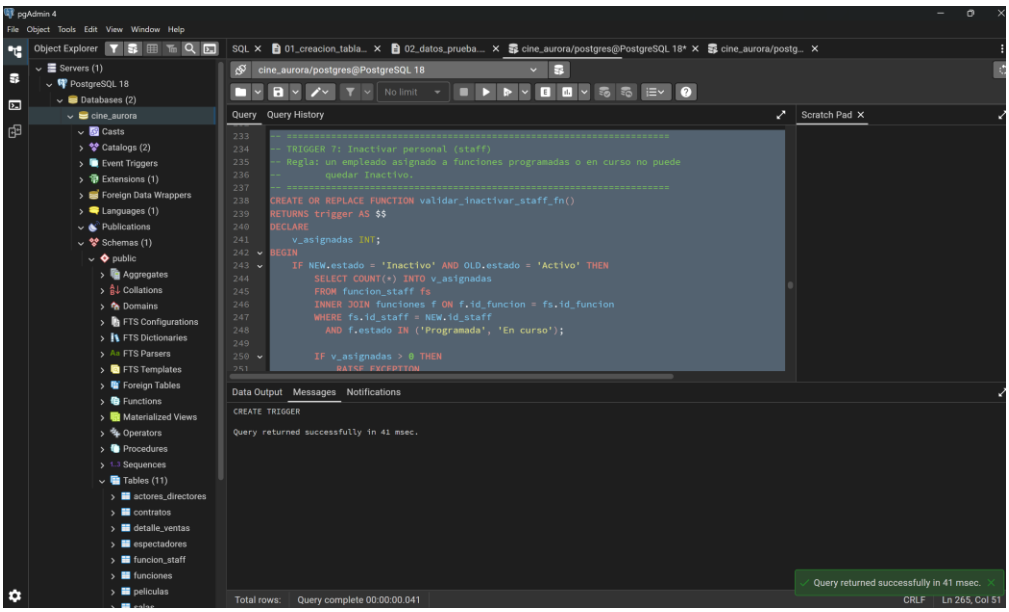


Trigger 7: trg_validar_inactivar_staff

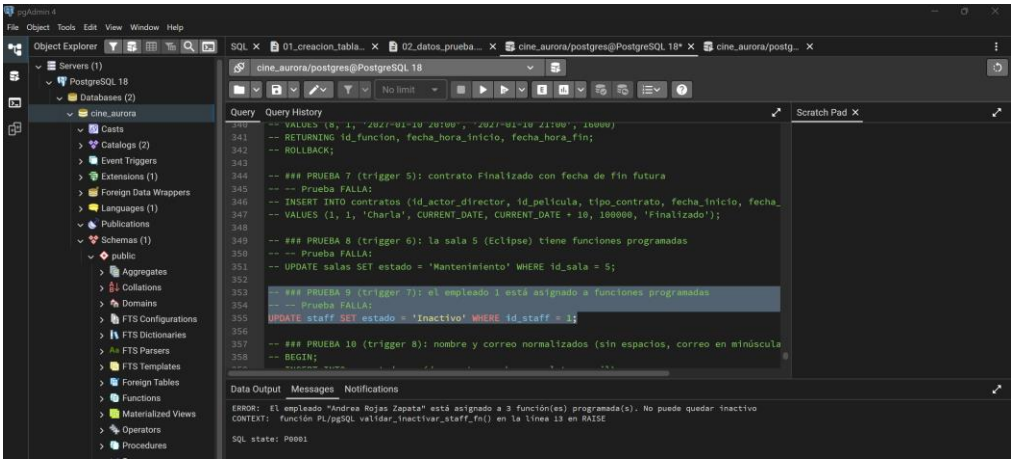
Tabla sobre la que actúa	staff
Evento	BEFORE UPDATE
Función asociada	validar_inactivar_staff_fn()
Regla de negocio que protege	No se puede inactivar a un empleado que está asignado a funciones programadas o en curso.

Cuando un empleado pasa de Activo a Inactivo, cuenta en cuántas funciones pendientes está asignado (tabla funcion_staff). Si tiene alguna, lanza un RAISE EXCEPTION para que esa función no se quede sin personal.

Captura 17. Ejecución del trigger 7 (trg_validar_inactivar_staff)



Captura 18. Prueba del trigger 7 (trg_validar_inactivar_staff)

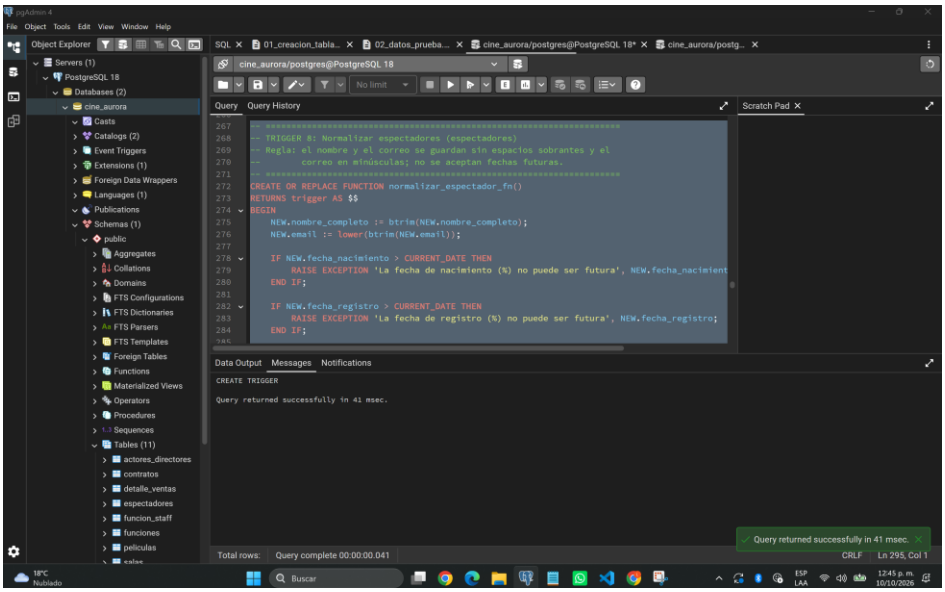


Trigger 8: trg_normalizar_espectador

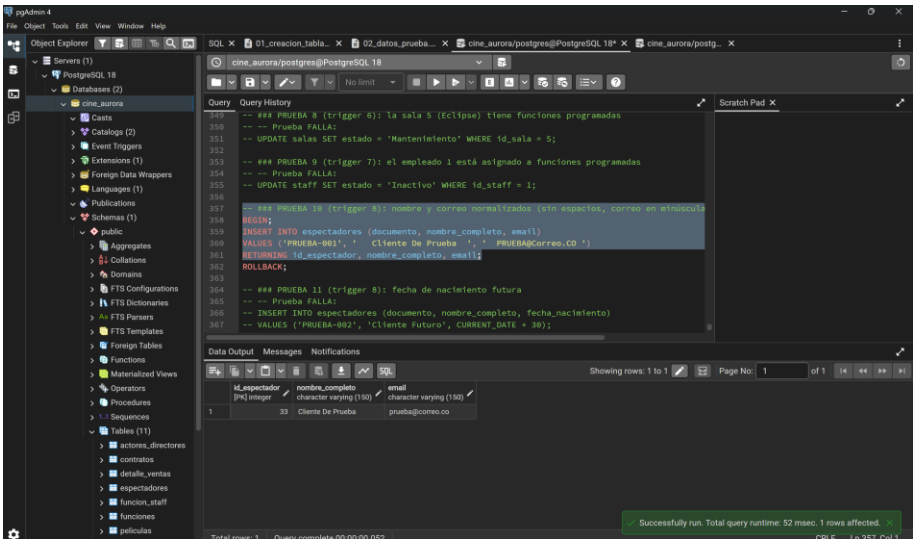
Tabla sobre la que actúa	espectadores
Evento	BEFORE INSERT OR UPDATE
Función asociada	normalizar_espectador_fn()
Regla de negocio que protege	Los datos del espectador deben quedar limpios y sin fechas futuras.

Quita los espacios que sobran en el nombre y el correo, pasa el correo a minúsculas (para que Ana@Correo.com y ana@correo.com no cuenten como correos distintos) y rechaza fechas de nacimiento o de registro que sean posteriores a hoy.ana@correo.com

Captura 19. Ejecución del trigger 8 (trg_normalizar_espectador)



Captura 20. Prueba del trigger 8 (trg_normalizar_espectador)

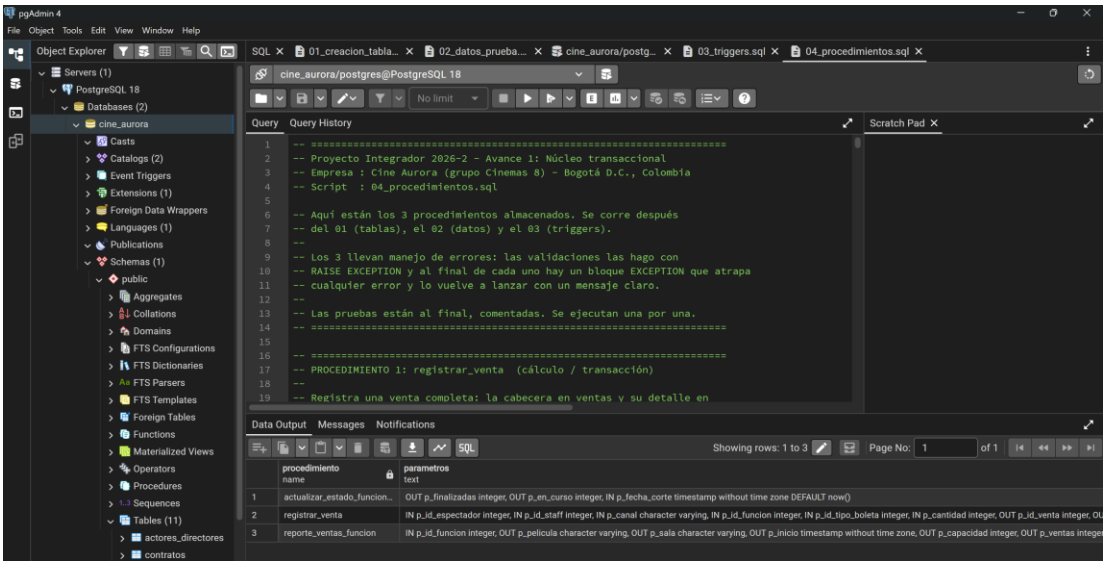


7. Procedimientos almacenados

El archivo 04_procedimientos.sql tiene 3 procedimientos, uno de cada tipo que piden: uno de cálculo o transacción, uno de reporte con agregación y uno de mantenimiento. Los tres validan con RAISE EXCEPTION y tienen un bloque EXCEPTION que atrapa cualquier error y lo vuelve a lanzar con un mensaje.

	Procedimiento	**Tipo**
1	registrar_venta	Cálculo / transacción
2	reporte_ventas_funcion	Reporte con agregación
3	actualizar_estado_funciones	Mantenimiento / operación

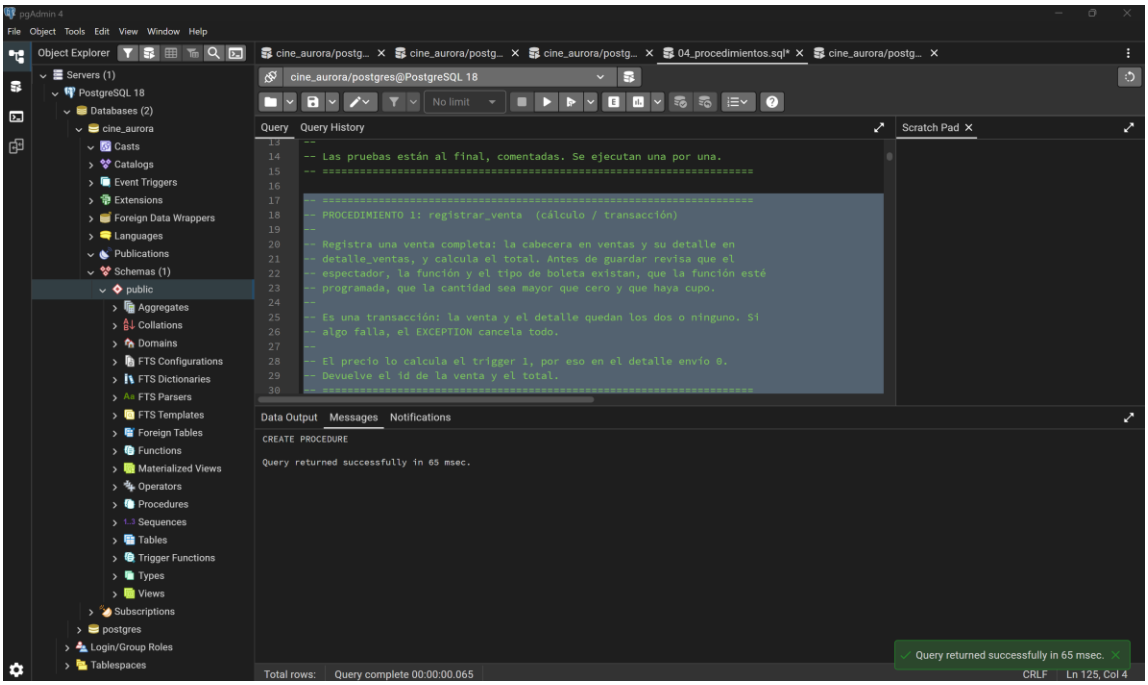
Captura 21. Ejecución del script 04_procedimientos.sql



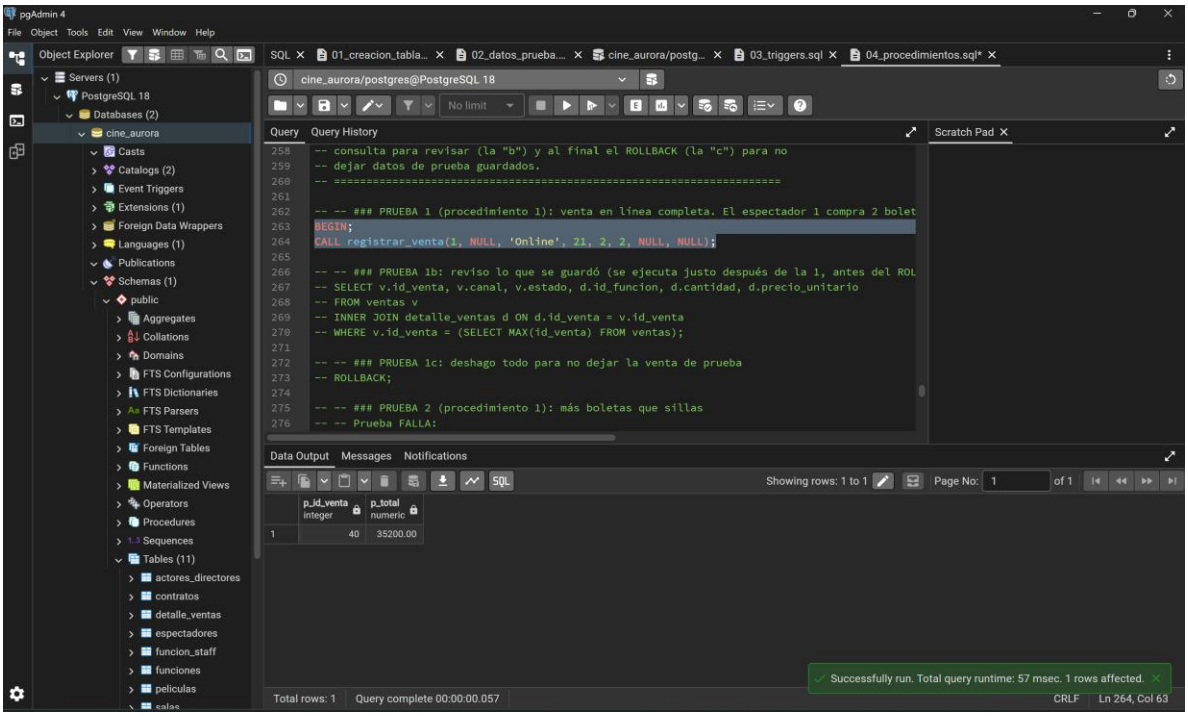
Procedimiento 1: registrar_venta

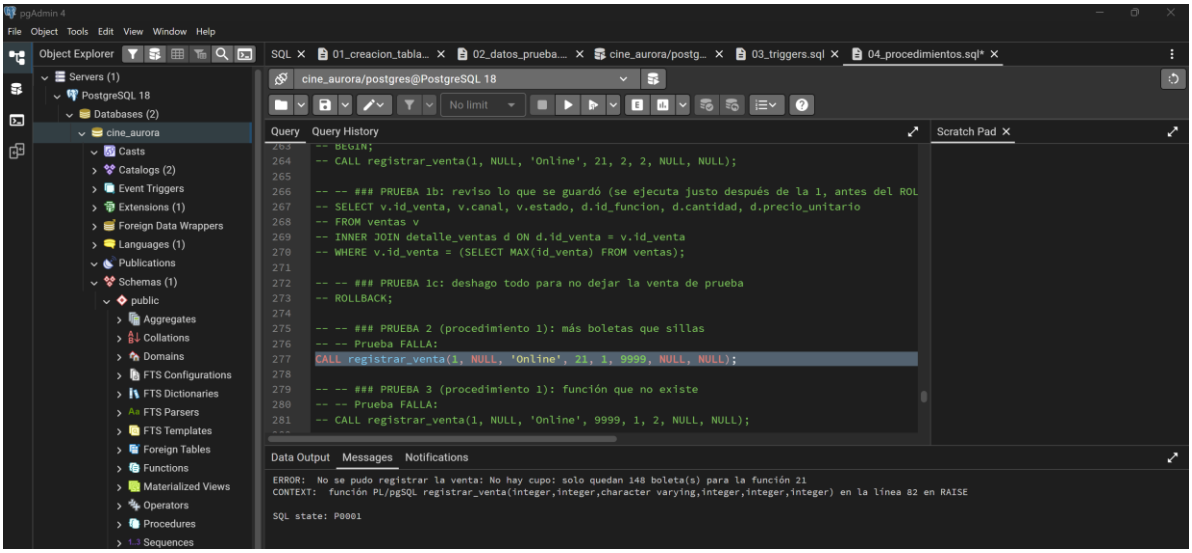
Tipo	Cálculo / transacción
Parámetros	Entran p_id_espectador, p_id_staff, p_canal, p_id_funcion, p_id_tipo_boleta y p_cantidad. Salen p_id_venta y p_total.
Propósito de negocio	Registra una venta completa: guarda la venta en ventas y su línea en detalle_ventas, y calcula el total (precio unitario por cantidad). Es una transacción, así que se guardan las dos cosas o no se guarda ninguna.
Validaciones	Que la cantidad sea mayor que cero, que el canal sea válido, que existan el espectador, el tipo de boleta y la función, que la función esté Programada y que haya cupo. Además, actúan los triggers 1 (precio), 2 (capacidad) y 3 (taquillero).
Manejo de excepciones / agregación	Cada validación usa RAISE EXCEPTION. El bloque EXCEPTION WHEN OTHERS cancela todo y lanza el error con el mensaje "No se pudo registrar la venta" seguido del motivo.

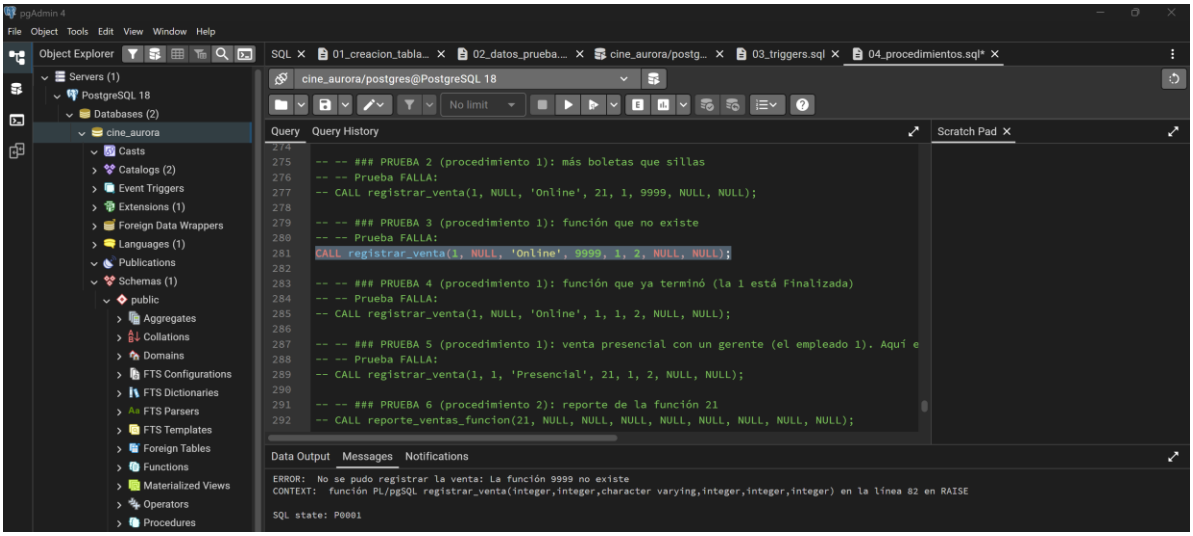
Captura 22. Ejecución del procedimiento 1 (registrar_venta)

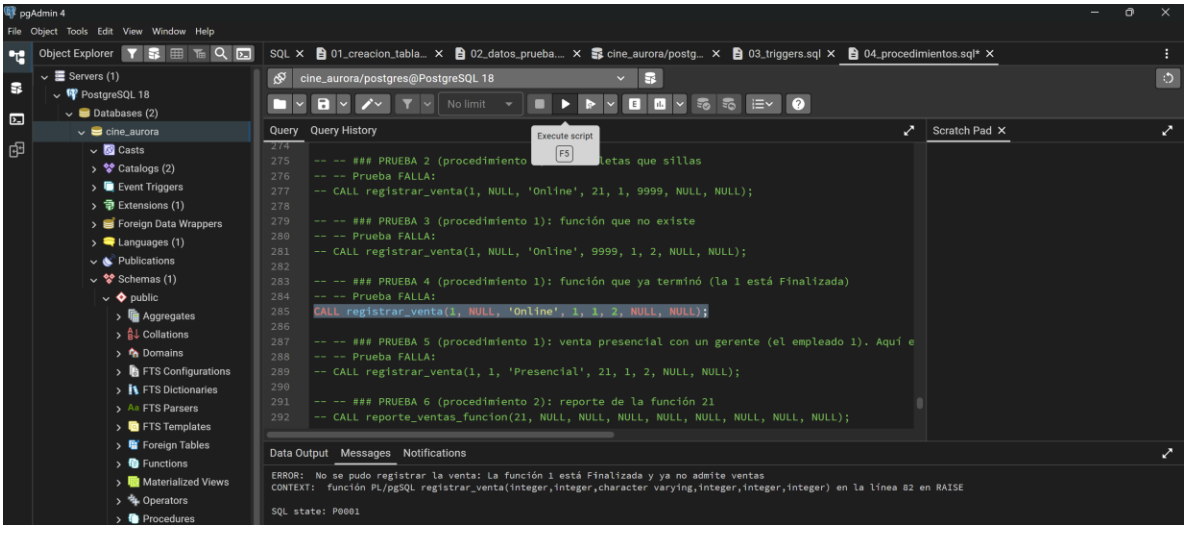


Captura 23. Pruebas del procedimiento 1 (registrar_venta)




pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
SQL 01_creacion_tabla... 02_datos_prueba.... cine_aurora/postg... 03_triggers.sql 04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
-- BEGIN;
-- CALL registrar_venta(1, NULL, 'Online', 21, 2, 2, NULL, NULL);
-- -- ### PRUEBA 1b: reviso lo que se guardó (se ejecuta justo después de la 1, antes del ROL
-- SELECT v.id_venta, v.canal, v.estado, d.id_funcion, d.cantidad, d.precio_unitario
-- FROM ventas v
-- INNER JOIN detalle_ventas d ON d.id_venta = v.id_venta
-- WHERE v.id_venta = (SELECT MAX(id_venta) FROM ventas);
-- -- ### PRUEBA 1c: deshago todo para no dejar la venta de prueba
-- ROLLBACK;
-- -- ### PRUEBA 2 (procedimiento 1): más boletas que sillas
-- -- Prueba FALLA:
CALL registrar_venta(1, NULL, 'Online', 21, 1, 9999, NULL, NULL);
-- -- ### PRUEBA 3 (procedimiento 1): función que no existe
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 9999, 1, 2, NULL, NULL);
Data Output Messages Notifications
ERROR: No se pudo registrar la venta: No hay cupo: solo quedan 148 boleta(s) para la función 21
CONTEXT: función PL/pgSQL registrar_venta(integer,integer,character varying,integer,integer,integer) en la línea 82 en RAISE
SQL state: P0001

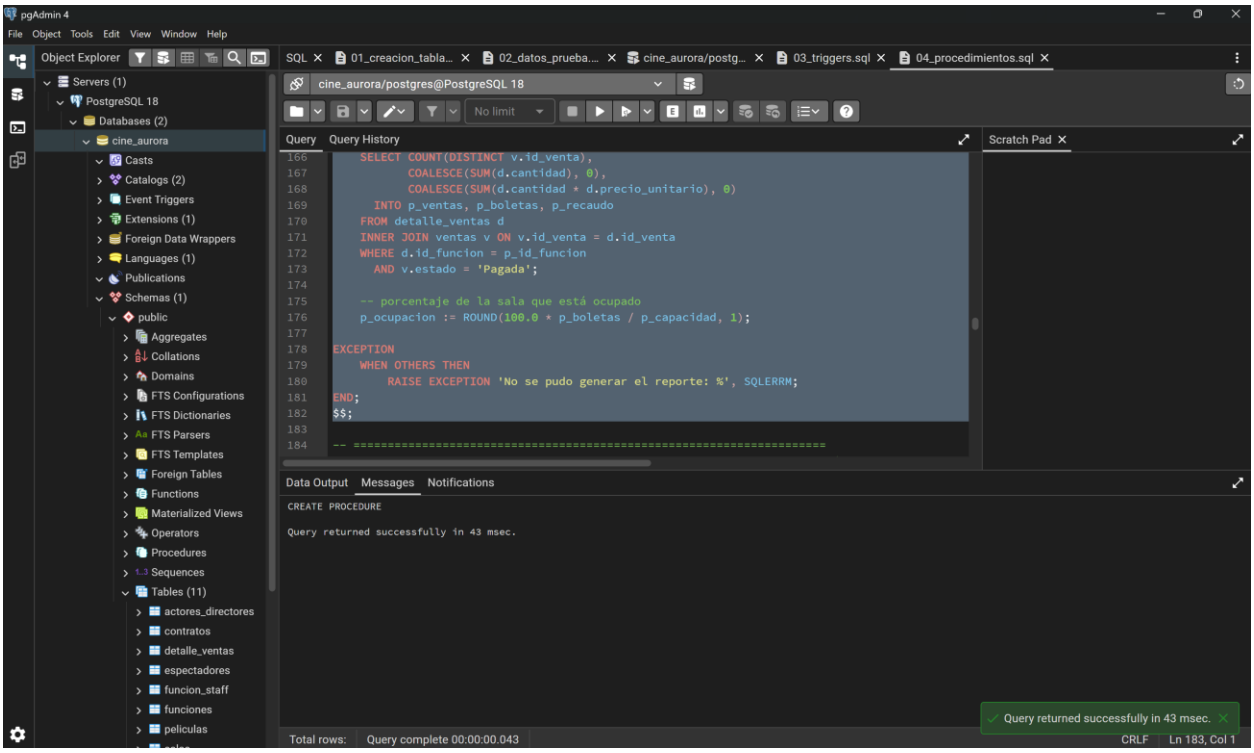

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
SQL 01_creacion_tabla... 02_datos_prueba.... cine_aurora/postg... 03_triggers.sql 04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
-- -- ### PRUEBA 2 (procedimiento 1): más boletas que sillas
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 21, 1, 9999, NULL, NULL);
-- -- ### PRUEBA 3 (procedimiento 1): función que no existe
-- -- Prueba FALLA:
CALL registrar_venta(1, NULL, 'Online', 9999, 1, 2, NULL, NULL);
-- -- ### PRUEBA 4 (procedimiento 1): función que ya terminó (la 1 está Finalizada)
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
-- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
-- -- Prueba FALLA:
-- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
-- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
-- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
Data Output Messages Notifications
ERROR: No se pudo registrar la venta: La función 9999 no existe
CONTEXT: función PL/pgSQL registrar_venta(integer,integer,character varying,integer,integer,integer) en la línea 82 en RAISE
SQL state: P0001


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
SQL 01_creacion_tabla... 02_datos_prueba.... cine_aurora/postg... 03_triggers.sql 04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Execute script
F5
Query Query History
Scratch Pad
-- -- ### PRUEBA 2 (procedimiento 1): más boletas que sillas
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 21, 1, 9999, NULL, NULL);
-- -- ### PRUEBA 3 (procedimiento 1): función que no existe
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 9999, 1, 2, NULL, NULL);
-- -- ### PRUEBA 4 (procedimiento 1): función que ya terminó (la 1 está Finalizada)
-- -- Prueba FALLA:
CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
-- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
-- -- Prueba FALLA:
-- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
-- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
-- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
Data Output Messages Notifications
ERROR: No se pudo registrar la venta: La función 1 está Finalizada y ya no admite ventas
CONTEXT: función PL/pgSQL registrar_venta(integer,integer,character varying,integer,integer,integer) en la línea 82 en RAISE
SQL state: P0001

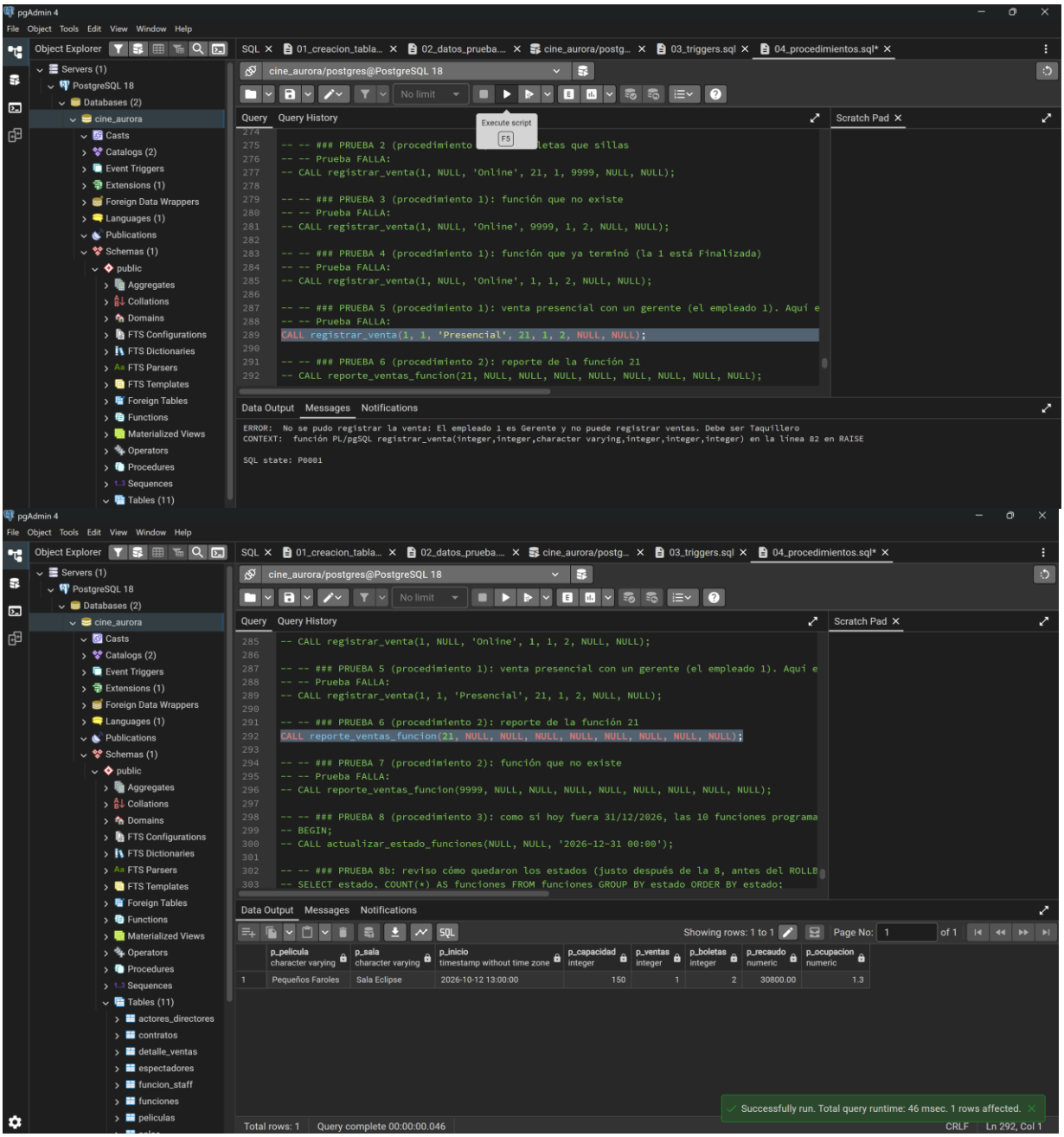
Procedimiento 2: reporte_ventas_funcion

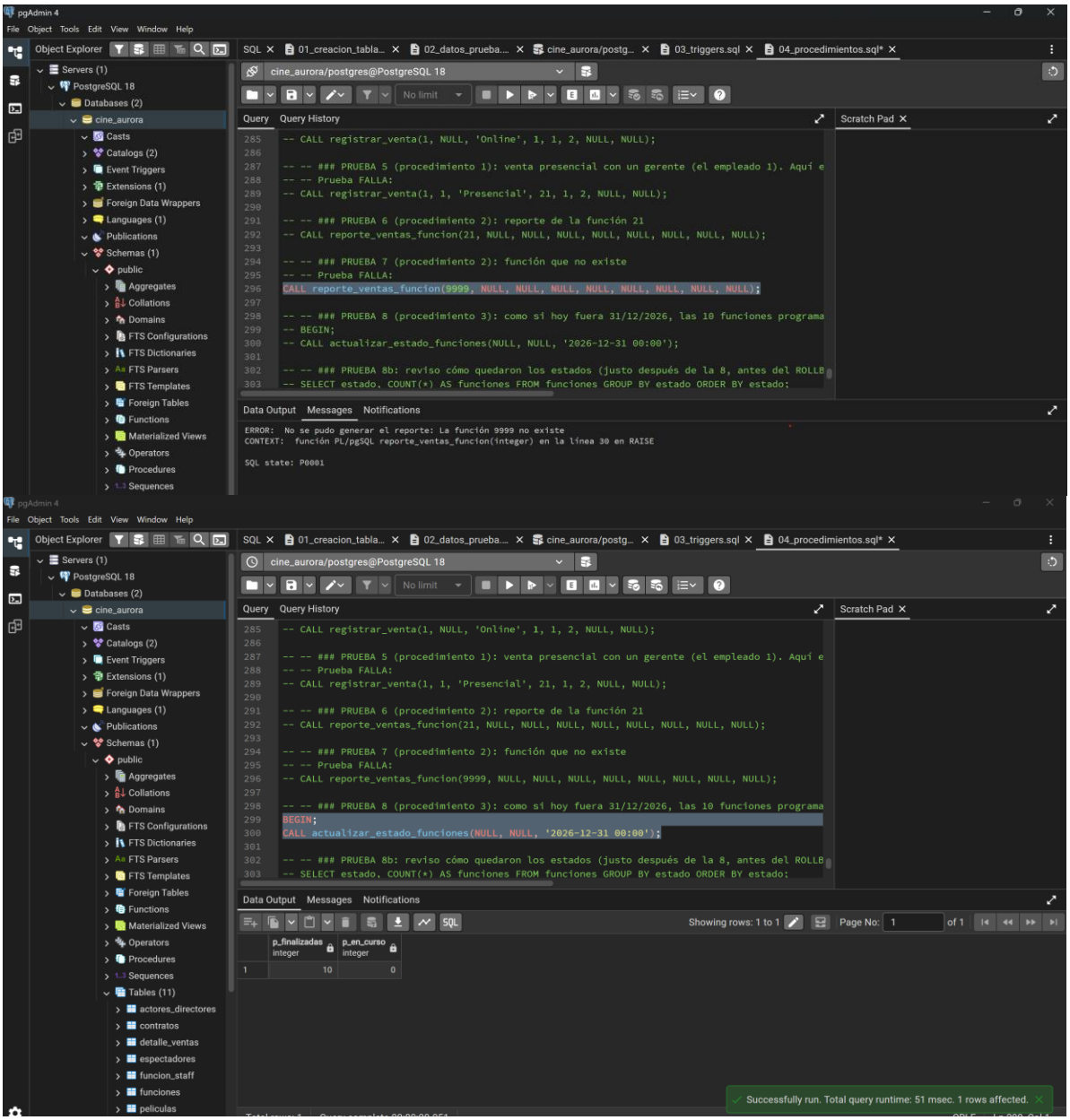
Tipo	Reporte con agregación
Parámetros	Entra p_id_funcion. Salen p_pelicula, p_sala, p_inicio, p_capacidad, p_ventas, p_boletas, p_recaudo y p_ocupacion.
Propósito de negocio	Hace el reporte de una función: la película, la sala, la hora de inicio, cuántas ventas tiene, cuántas boletas se vendieron, cuánto se recaudó y qué porcentaje de la sala está ocupado.
Validaciones	Que la función exista. Solo se cuentan las ventas Pagadas, las anuladas no suman.
Manejo de excepciones / agregación	Usa COUNT (DISTINCT id_venta), SUM (cantidad) y SUM (cantidad * precio_unitario). Si hay un error, el bloque EXCEPTION lo lanza otra vez con el mensaje "No se pudo generar el reporte" seguido del motivo.

Captura 24. Ejecución del procedimiento 2 (reporte_ventas_funcion)



Captura 25. Pruebas del procedimiento 2 (reporte_ventas_funcion)



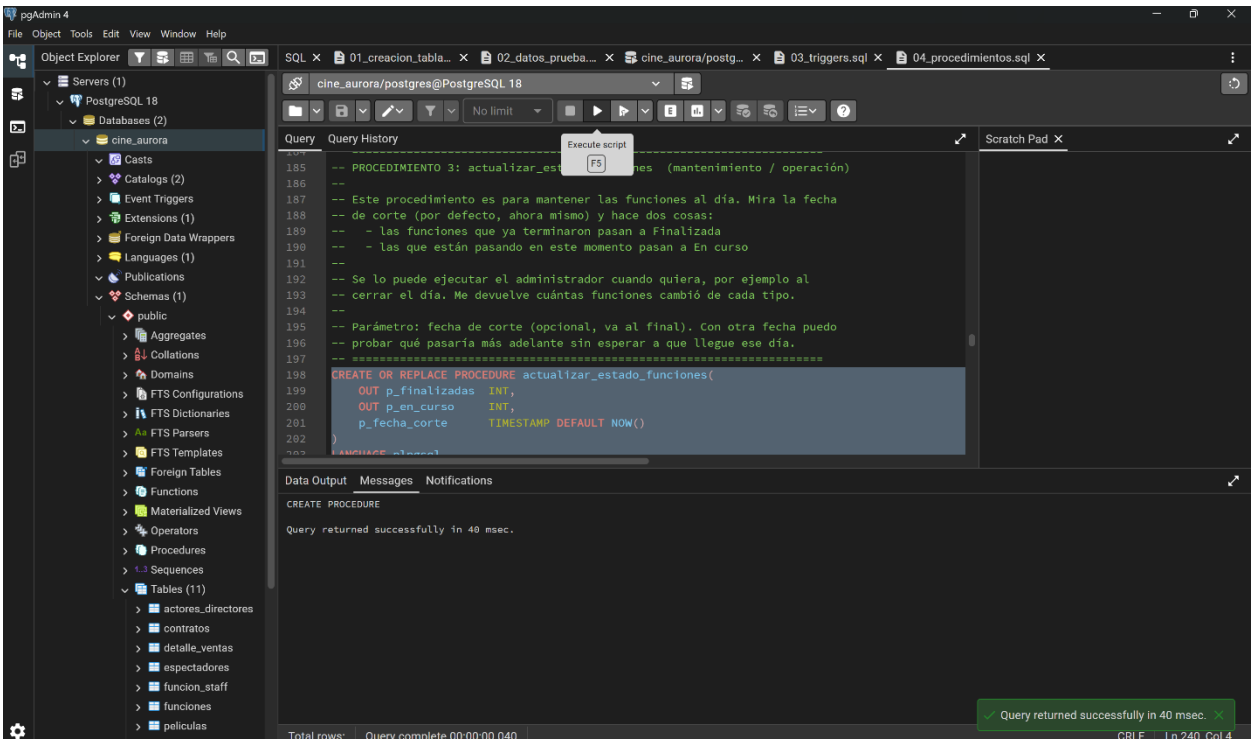


Procedimiento 3: actualizar_estado_funciones

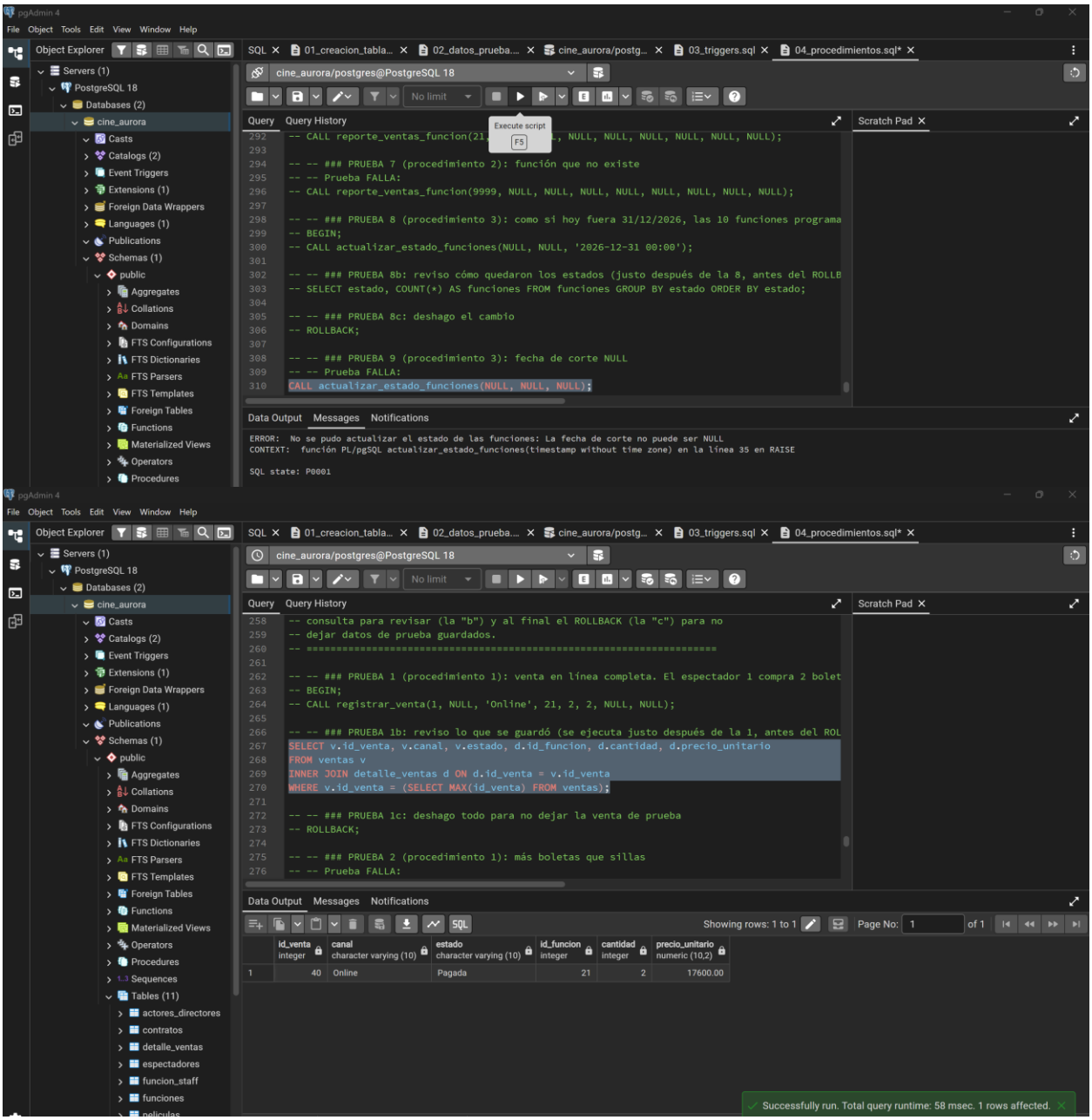
Tipo	Mantenimiento / operación

Parámetros	Entra p_fecha_corte (es opcional y por defecto es NOW ()). Salen p_finalizadas y p_en_curso.
Propósito de negocio	Mantiene al día el estado de las funciones según una fecha de corte: las que ya terminaron pasan a Finalizada y las que están pasando en ese momento pasan a En curso. Devuelve cuántas cambió de cada tipo. Si se usa otra fecha de corte se puede probar qué pasaría más adelante.
Validaciones	Que la fecha de corte no sea NULL.
Manejo de excepciones / agregación	Lanza un RAISE EXCEPTION si la fecha es nula, y el bloque EXCEPTION vuelve a lanzar cualquier error con el mensaje "No se pudo actualizar el estado de las funciones" seguido del motivo.

Captura 26. Ejecución del procedimiento 3 (actualizar_estado_funciones)



Captura 27. Pruebas del procedimiento 3 (actualizar_estado_funciones)



8. Consultas SQL

El archivo 05_consultas.sql tiene 3 consultas que responden preguntas que se haría la administración del cine. Las tres usan JOIN, funciones de agregación y subconsultas.

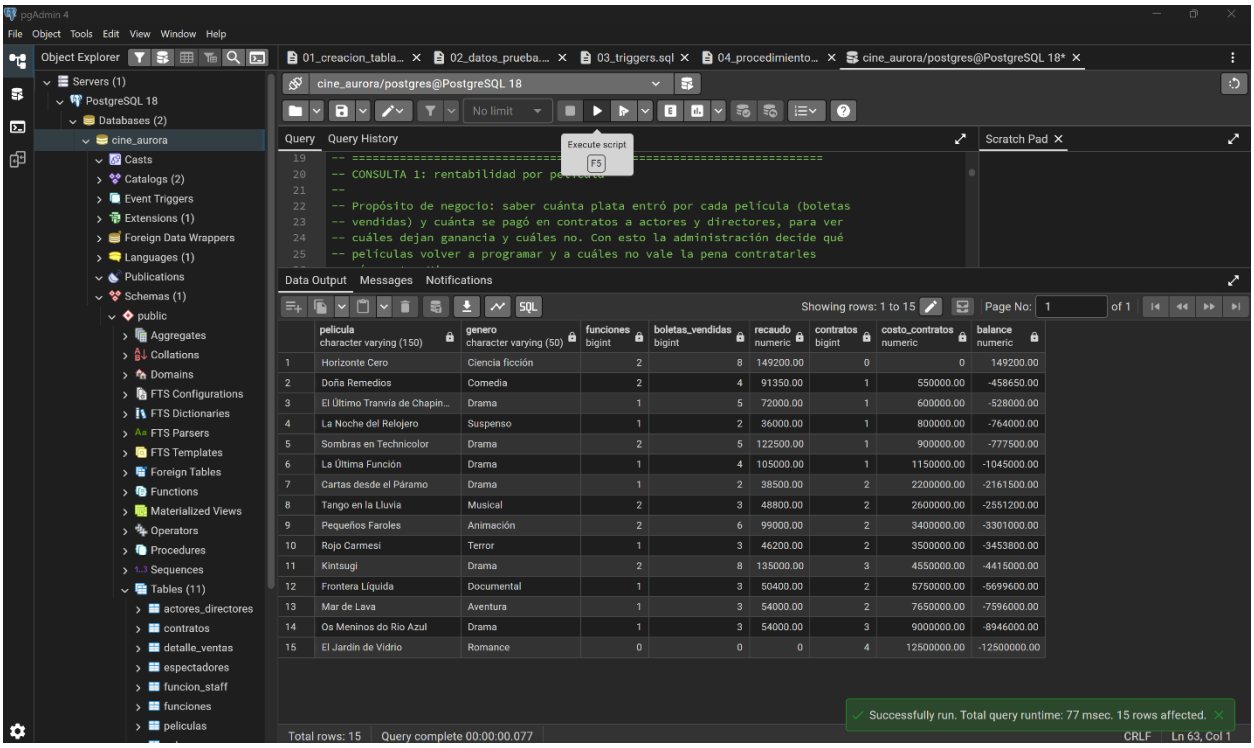
Consulta 1: Rentabilidad por película

Pregunta de negocio	¿Qué películas dejan más dinero y cuáles no pagan lo que cuestan?
Propósito de negocio	Compara lo que se recaudó con las boletas de cada película contra lo que se pagó en contratos a actores y directores. Sirve para decidir qué películas conviene volver a programar y con cuáles no vale la pena seguir contratando artistas.

Cómo está construida:

- Una subconsulta en el FROM suma el recaudo por película (cantidad por precio_unitario), solo de ventas Pagadas.
- Otra subconsulta suma el valor de los contratos que no están cancelados, por película.
- Se unen con LEFT JOIN a películas para que también salgan las películas sin ventas o sin contratos (con 0).
- Las dos sumas se hacen por separado para que las filas no se multipliquen al unir las tablas.
- El balance es el recaudo menos el costo de los contratos, ordenado de mayor a menor.

Captura 28. Consulta 1, código y resultado



	película	genero	funciones	boletas_vendidas	recaudo	contratos	costo_contratos	balance
1	Horizonte Cero	Ciencia ficción	2	8	149200.00	0	0	149200.00
2	Doña Remedios	Comedia	2	4	91350.00	1	550000.00	-458650.00
3	El Último Tranvía de Chapin…	Drama	1	5	72000.00	1	600000.00	-528000.00
4	La Noche del Relojero	Suspenso	1	2	36000.00	1	800000.00	-764000.00
5	Sombras en Technicolor	Drama	2	5	122500.00	1	900000.00	-777500.00
6	La Última Función	Drama	1	4	105000.00	1	1150000.00	-1045000.00
7	Cartas desde el Páramo	Drama	1	2	38500.00	2	2200000.00	-2161500.00
8	Tango en la Lluvia	Musical	2	3	48800.00	2	2600000.00	-2551200.00
9	Pequeños Faroles	Animación	2	6	99000.00	2	3400000.00	-3301000.00
10	Rojo Carmesí	Terror	1	3	46200.00	2	3500000.00	-3453800.00
11	Kintsugi	Drama	2	8	135000.00	3	4550000.00	-4415000.00
12	Frontera Líquida	Documental	1	3	50400.00	2	5750000.00	-5699600.00
13	Mar de Lava	Aventura	1	3	54000.00	2	7650000.00	-7596000.00
14	Os Meninos do Rio Azul	Drama	1	3	54000.00	3	9000000.00	-8946000.00
15	El Jardín de Vidrio	Romance	0	0	0	4	12500000.00	-12500000.00

Interpretación de los resultados: De las 15 películas, solo Horizonte Cero tiene balance positivo $149.200. Las otras 14 quedan en negativo porque lo que se pagó en contratos es mucho más que lo recaudado con las boletas. El caso más extremo es El Jardín de Vidrio, que no tiene funciones ni boletas

vendidas, pero sí 4 contratos, por eso su balance es de -$12.500.000. Parte de esto se debe a que los datos de prueba tienen pocas ventas, pero igual permite ver qué películas no están pagando lo que cuestan. Con este resultado la administración podría dejar de contratar artistas para las películas con peor balance y programar más veces Horizonte Cero.

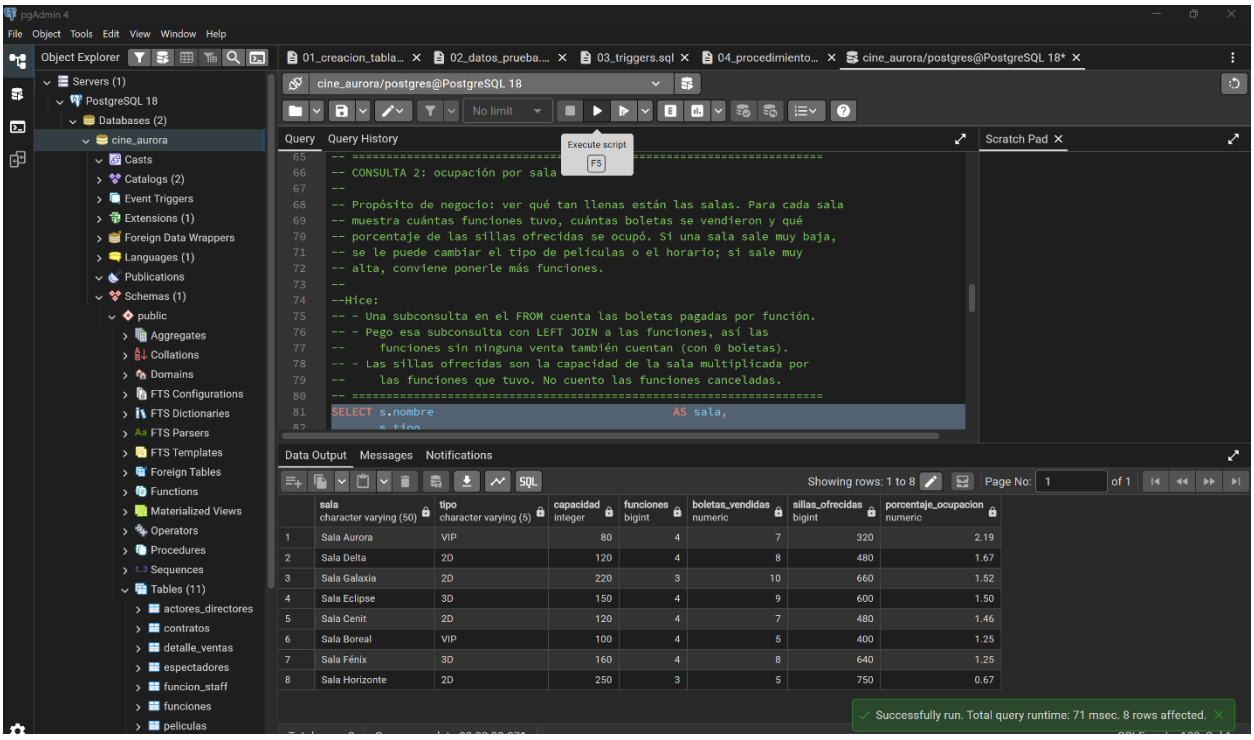
Consulta 2: Ocupación por sala

Pregunta de negocio	¿Qué salas se están llenando y cuáles quedan vacías?
Propósito de negocio	Muestra por sala cuántas funciones tuvo, cuántas boletas se vendieron y qué porcentaje de las sillas ofrecidas se ocupó. Si la ocupación es baja se puede cambiar el tipo de películas o el horario, y si es muy alta conviene ponerle más funciones.

Cómo está construida:

- Una subconsulta en el FROM cuenta las boletas pagadas por función.
- Se une con LEFT JOIN a funciones para que también cuenten las funciones sin ventas (0 boletas).
- Las sillas ofrecidas son la capacidad de la sala por el número de funciones, sin contar las canceladas.
- Se agrupa por sala y el porcentaje de ocupación se calcula con ROUND.

Captura 29. Consulta 2, código y resultado



Interpretación de los resultados: La consulta muestra la ocupación de las 8 salas y en todas es muy baja, ninguna llega al 2,2 % de las sillas ofrecidas. La Sala Aurora VIP, 80 sillas es la más ocupada con 2,19 % 7 boletas en 4 funciones, seguida de la Sala Delta con 1,67 % y la Sala Galaxia con 1,52 %. La menos ocupada es la Sala Horizonte 2D, 250 sillas con 0,67 %, porque es la más grande y solo vendió 5 boletas

en 3 funciones. Con esto la administración vería que por ahora no conviene agregar más funciones, sino revisar los horarios y las películas, hacer promociones y programar las películas con menos público en salas más pequeñas. Los porcentajes salen bajos porque los datos de prueba tienen pocas ventas 59 boletas en total

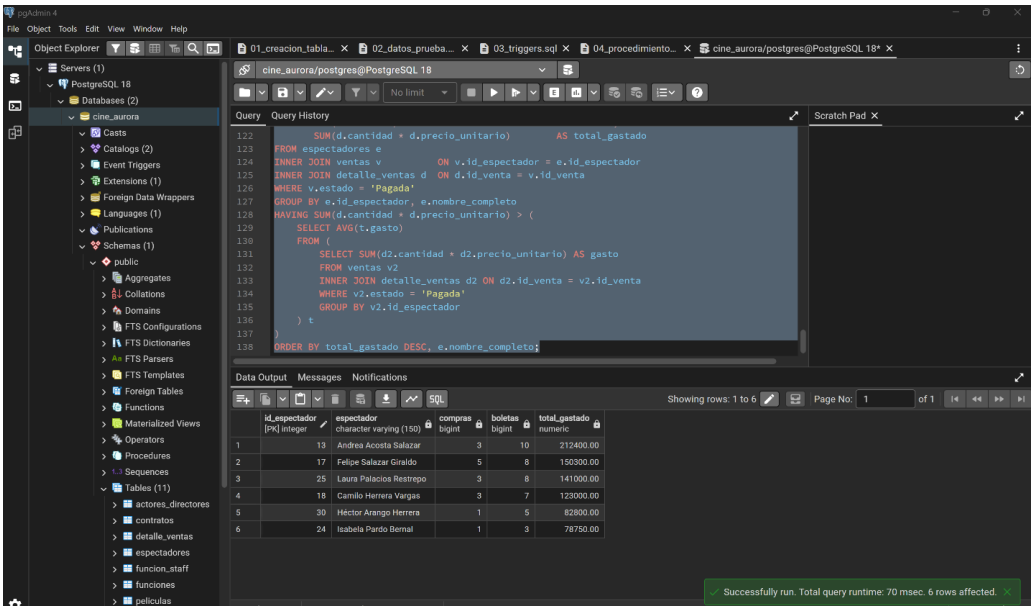
Consulta 3: Espectadores que gastan más que el promedio

Pregunta de negocio	¿Quiénes son los clientes más valiosos?
Propósito de negocio	Busca a los espectadores que han gastado más que el promedio de los que compraron, para ofrecerles promociones o un programa de fidelidad.

Cómo está construida:

- Se unen espectadores, ventas y detalle_ventas, solo con ventas Pagadas.
- Se agrupa por espectador para sacar sus compras, sus boletas y el total gastado.
- El HAVING compara ese total con una subconsulta que calcula el promedio de gasto. Se usa HAVING y no WHERE porque se compara contra una suma.

Captura 30. Consulta 3, código y resultado



Interpretación de los resultados: La consulta devolvió 6 espectadores que gastaron más que el promedio de los clientes que compraron. Andrea Acosta Salazar es la que más gastó, $212.400 en 10 boletas, seguida de Felipe Salazar Giraldo $150.300, que además es quien más compras hizo 5, y de Laura Palacios Restrepo $141.000. Héctor Arango Herrera $82.800 e Isabela Pardo Bernal $78.750 solo hicieron una compra, pero con varias boletas. Con esta lista la administración podría ofrecer un programa de fidelidad a estos 6 clientes y un incentivo de recompra a los que compraron una sola vez.

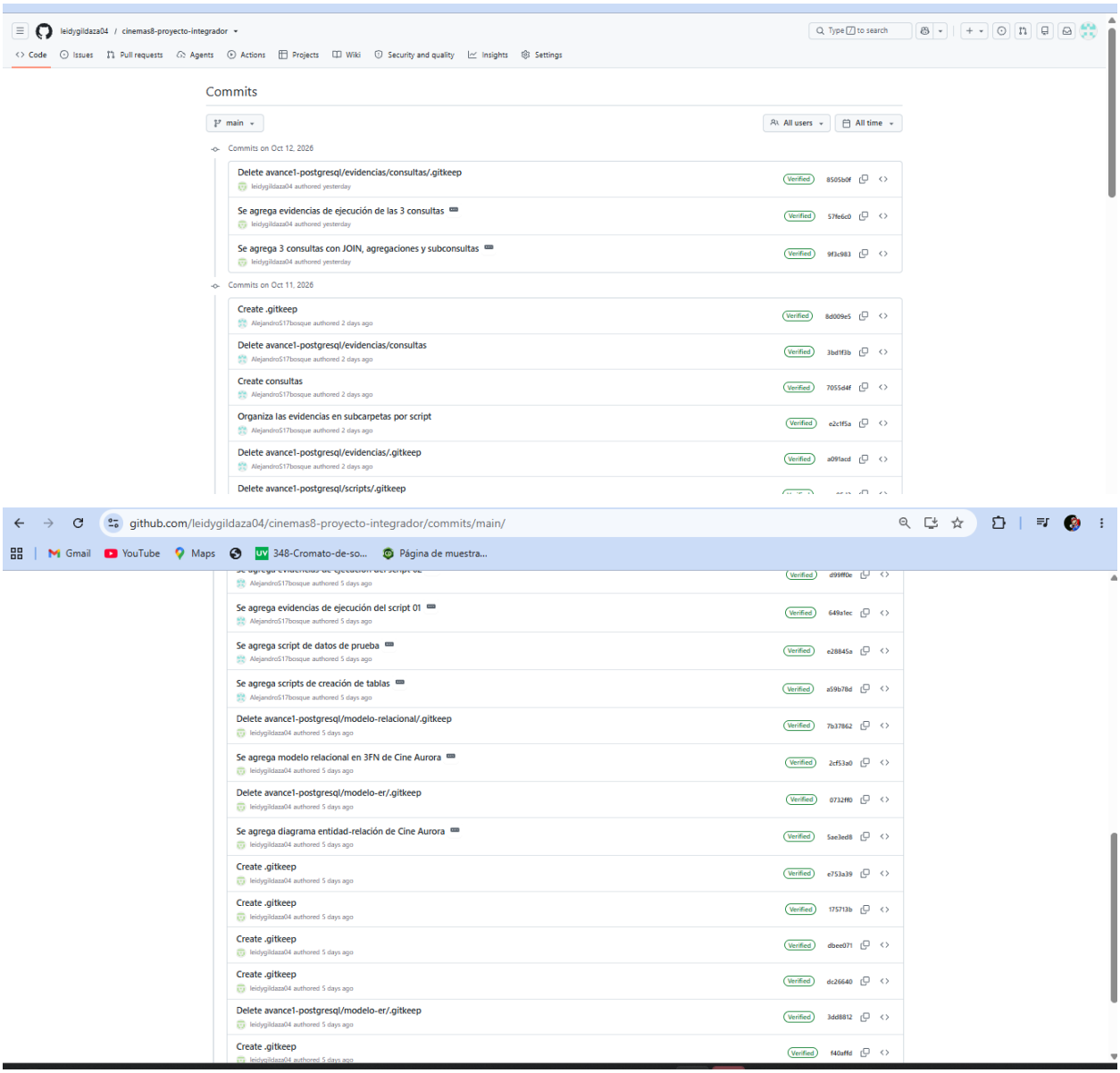
9. Repositorio y control de versiones

El trabajo está en un repositorio de GitHub, en la carpeta avance1-postgresql/, con estos archivos: Aquí tienes la tabla exacta en formato Markdown, lista para copiar y pegar en tu documento:

Archivo	Contenido
evidencias/	Capturas de ejecución organizadas por subcarpetas (consultas, creación de tablas, datos de prueba, procedimientos y triggers).
modelo-er/	Carpeta que contiene el diagrama y archivo del Modelo Entidad-Relación (cinemas8_modelo_er.pdf).
modelo-relacional/	Carpeta que contiene el diagrama y archivo del Modelo Relacional Normalizado en 3FN (cinemas8_modelo_relacional_3fn.pdf).
scripts/	Carpeta que contiene los scripts SQL del proyecto:
└─ 01_creacion_tablas.sql	Creación de las 11 tablas, llaves, restricciones e índices.
└─ 02_datos_prueba.sql	Datos de prueba coherentes.
└─ 03_triggers.sql	8 triggers con sus respectivas pruebas.
└─ 04_procedimientos.sql	3 procedimientos almacenados con sus pruebas.
└─ 05_consultas.sql	3 consultas SQL.
README.md	Archivo con la descripción general y documentación del repositorio.
avance1-informe.pdf	Este informe final en formato PDF.

Enlace al repositorio	https://github.com/leidygildaza04/cinemas8-proyecto-integrador/tree/main/avance1-postgresql

9.1 Commits por integrante



Captura 18. Estructura de la carpeta avance1-postgresql/ en GitHub

